KU LEUVEN

FACULTEIT
INGENIEURSWETENSCHAPPEN

Nieuwe aanpakken voor de Amazon Last Mile Routing Research Challenge

Alessandro Marchese

Thesis voorgedragen tot het behalen
van de graad van Master of Science
in de ingenieurswetenschappen:
Computerwetenschappen,
optie Artificiële Intelligentie

Promotor:
Prof. dr. Tias Guns

Assessoren:
Dr. ir. Samuel Kolb
Prof. dr. ir. Jan Goedgebeur

Begeleiders:
Rocsildes Canoy
Bastián Véjar

Academiejaar 2022-2023

© Copyright by KU Leuven

Without written permission of the supervisor(s) and the authors it is forbidden to reproduce or adapt in any form or by any means any part of this publication. Requests for obtaining the right to reproduce or utilize parts of this publication should be addressed to KU Leuven, Faculty of Engineering Science - Kasteelpark Arenberg 1, B-3001 Heverlee (Belgium). Telephone +32-16-32 13 50 & Fax. +32-16-32 19 88.

A written permission of the supervisor(s) is also required to use the methods, products, schematics and programs described in this work for industrial or commercial use, and for submitting this publication in scientific contests.

© Copyright by KU Leuven

Zonder voorafgaande schriftelijke toestemming van zowel de promoter(en) als de auteur(s) is overnemen, kopiëren, gebruiken of realiseren van deze uitgave of gedeelten ervan verboden. Voor aanvragen tot of informatie i.v.m. het overnemen en/of gebruik en/of realisatie van gedeelten uit deze publicatie, wend u tot de KU Leuven, Faculteit Ingenieurswetenschappen - Kasteelpark Arenberg 1, B-3001 Heverlee (België). Telefoon +32-16-32 13 50 & Fax. +32-16-32 19 88.

Voorafgaande schriftelijke toestemming van de promoter(en) is eveneens vereist voor het aanwenden van de in dit afstudeerwerk beschreven (originele) methoden, producten, schakelingen en programma's voor industrieel of commercieel nut en voor de inzending van deze publicatie ter deelname aan wetenschappelijke prijzen of wedstrijden.

Voorwoord

Graag wil ik mijn vriendin, familie, begeleiders en promotor bedanken.

Alessandro Marchese

Inhoudsopgave

Voorwoord ..... **i**
Abstract ..... **iv**
Samenvatting ..... **v**
Lijst van figuren en tabellen ..... **vi**
Lijst van figuren vi
Lijst van tabellen vii
Lijst van afkortingen en symbolen ..... **viii**
Hoofdstuk 1: Introductie **1**
1.1 Het belang van e-commerce en last-mile bezorgingen 1
1.2 De 2021 Amazon Last Mile Routing Challenge 1
1.3 Verdere hoofdstukken en onderzoeksvragen 5
Hoofdstuk 2: Literatuurstudie **8**
2.1 Oorsprong van het TSP en oplossingen 8
2.2 Algemene informatie over de challenge en dataset 11
2.3 Eerdere aanpakken en dataset analyses 13
2.7 Maximum Likelihood Routing en het 2-staps algoritme 21
2.8 Het gebruik van Neurale Netwerken 28
2.9 Conclusie literatuurstudie 31
Hoofdstuk 3: Methodologie **33**
3.2 Theoretische experimenten 33
3.3 Andere manieren van tellen 34
3.4 Referentie mechanisme 37
3.5 NN voor distributies te veralgemenen 39
3.6 NN voor lineaire combinaties van distributies 43
Hoofdstuk 4: Numerieke resultaten **45**
4.1 Theoretische experimenten 45
4.2 Experimenten op manieren van tellen 46

4.3 Lineaire combinaties van tellen en distributies 48
4.4 Lineaire combinaties voor elke source 50
4.5 Veralgemenen van distributies 52
4.6 Conclusie en reflectie uit experimenten 55
Hoofdstuk 5: Suggesties voor toekomstig onderzoek 57
5.1 Input variabelen 57
5.2 Referentie mechanisme 57
5.3 Generatieve modellen 58
5.4 Anders trainen 58
5.5 Aanpassingen voor challenge 59
Hoofdstuk 6: Conclusie 60
Bibliografie 62

Abstract

In this thesis, the Amazon Last Mile Routing Challenge is first thoroughly examined based on traditional methods, dataset analyses, and the algorithms of the winning teams. The study builds upon the Maximum Likelihood Routing framework presented by Canoy and Guns (2019).

Theoretical tests conducted on the existing two-step algorithm by Canoy et al. (2022) indicate that, if the second step were to achieve a perfect zone order, it could achieve a stop score of 0.0079. This would nearly solve the Last Mile Challenge problem. However, the experiments do not prove this to be a feasible outcome. The goal of this study is to get as close to this result as possible by predicting the best possible zone order.

Firstly, new methods of Markov counting that are more specific to the given dataset are introduced. Some methods employ data augmentation by considering routes in both directions. A reference mechanism is also used to better determine the directions of proposed orders. The results show that this combination is an effective approach to dealing with the limited amount of data.

The study then attempts to predict the utility of transitions using Alternative Specific Utility DNNs, with the aim of determining a probability distribution across possible transitions. The problem is treated as a classification issue, with categorical cross-entropy used as the loss function. The best stop score achieved through this method is 0.0257.

Some variations of the architecture proposed in Mandi et al. (2021) are also considered. These architectures are based on using separate linear layers for each different source of transition. These resulted in poor results due to lack of data.

Future research may focus on better ways to determine directions of zone sequences such as including feature variables about the package sizes and their delivery times. For the learning process, the EM algorithm can be employed to more closely approximate the actual probability distributions during training. The use of other neural network architectures such as pointer networks and other learning paradigms such as reinforcement learning might also be explored.

Samenvatting

In dit werk wordt de Amazon Last Mile Routing Challenge eerst grondig geanalyseerd op basis van traditionele methodes, dataset analyses en de algoritmen van de winnende teams. Vervolgens wordt er verder gebouwd op het Maximum Likelihood Routing framewerk uit Canoy en Guns (2019).

Uit enkele theoretische testen uitgevoerd op het bestaand twee staps algoritme van Canoy et al. (2022) bleek dat, indien de tweede stap een perfecte zone volgorde zou krijgen, het een stop score van 0.0079 kan bereiken. Dit zou het probleem van de Last Mile Challenge bijna volledig oplossen. De experimenten bewijzen echter niet dat dit een haalbaar resultaat is. Het doel van dit werk is om zo dicht mogelijk bij deze uitkomst te komen door een zo goed mogelijke zone volgorde te voorspellen.

Er worden eerst nieuwe manieren van Markov counting voorgesteld die specifieker zijn voor de gegeven dataset. Sommige methodes maken gebruik van data augmentatie door routes in beide richtingen te beschouwen. Er wordt ook een referentie mechanisme gebruikt om de richtingen van voorgestelde volgorden beter te bepalen. Uit de resultaten blijkt dat deze combinatie een goede aanpak is om met de weinige hoeveelheid data om te gaan.

Vervolgens wordt er geprobeerd om aan de hand van Alternative Specific Utility DNN's de utiliteit van transities te voorspellen om dan aan de hand van de utiliteiten een kansendistributie over de mogelijke transities te bepalen. Het probleem wordt gezien als een classificatie probleem waarbij categorical cross-entropy als loss functie wordt gebruikt. De beste stop score dat hiermee bereikt wordt is van 0.0257.

Enkele variaties van de architectuur voorgesteld in Mandi et al. (2021) zullen ook worden beschouwd. Deze architecturen zijn gebaseerd op het gebruik van aparte linear layers voor elke verschillende bron van een transitie. Deze resulteerden in slechte resultaten door het gebrek aan data.

Toekomstig onderzoek kan zich focussen op het beter bepalen van richtingen van zone volgorden bijvoorbeeld door informatie te gebruiken over de groottes van pakketten en hun levertijden. Voor het leerproces kan het EM algoritme gebruikt worden om de echte kansen distributies beter te benaderen. Het gebruik van andere neuraal netwerk architecturen zoals pointer netwerken en andere leer paradigma's zoals reinforcement learning kan ook getest worden.

Lijst van figuren en tabellen

Lijst van figuren

Figuur 1 : Voorgestelde route VS bezorger 2

Figuur 2 : Voorbeeldroute bezorger (links) VS Google OR Tools (rechts) 3

Figuur 3 : Zakken per zone in Amazon camionet 4

Figuur 4 : Pakket voor stop in zone E-7.1G 4

Figuur 5 : Gerangschikte inzendingen voor de routing challenge 5

Figuur 6 : Histogram beste inzendingen voor de routing challenge 5

Figuur 7 : Station (rood) en cluster van stops (zwart) voor route#0002 18

Figuur 8 : Volgorde van Zone ID's in waargenomen tours 21

Figuur 9 : Visualisatie van 2-staps methode 25

Figuur 10 : Diagram van 2-staps methode 28

Figuur 11 : Aantal zone ID's per station ID en station ID's per stedelijke ruimte 34

Figuur 12 : Alle stops bereikt door DSE4 ingekleurd per zone ID (links) en de zone ID's geclusterd op basis van hun eerste symbool (rechts) 35

Figuur 13 : Alle stops in het stedelijk gebied van Seattle. Verschillende kleuren worden gebruikt om het bereik van verschillende stations te illustreren 36

Figuur 14 : Voorbeeld route om te evalueren aan de hand van referentie (start: links beneden in D-5.1E) 39

Figuur 15 : Referentie route voor voorbeeld route (start: links beneden in D-5.2E) 39

Figuur 16 : Architectuur gebruikt om kans distributies te veralgemenen 41

Figuur 17 : Architectuur gebruikt voor lineaire combinaties van distributies te nemen 43

Lijst van tabellen

Tabel 1 : Resultaten experimenten op verschillende manieren van tellen 47

Tabel 2 : Resultaten experimenten op verschillende manieren van tellen met afstanden .. 47

Tabel 3 : Linear Layer experimenten met ideale connectie kansverdelingen 49

Tabel 4 : Linear layer experimenten met ideale transitie kansverdelingen 49

Tabel 5 : Collectieve training van linear layers waarbij source bepaald wordt door zone ID 50

Tabel 6 : Individuele training van linear layers waarbij source bepaald wordt door zone ID 50

Tabel 7 : Stop scores voor linear layer model met source bepaald door zone ID 51

Tabel 8 : Collectieve training van linear layers waarbij source bepaald wordt door unieke ID 52

Tabel 9 : Individuele training van linear layers waarbij source bepaald wordt door unieke ID 52

Tabel 10 : Stop scores voor linear layer model met source bepaald door unieke ID 52

Tabel 11 : Resultaten op globale connectie targets afhankelijk van het aantal parameters 53

Tabel 12 : Resultaten van generalisatie van globale en stedelijke tellen 53

Tabel 13 : Resultaten van generalisatie van globale connecties, station transities en afstanden 54

Tabel 14 : Resultaten van generalisatie van ideale connectie en transitie distributies met hyperparameter tuning 54

Tabel 15 : Resultaten van generalisatie van ideale connectie en transitie distributies met hyperparameter tuning met 6 initiële distributies als inputs 55

Lijst van afkortingen en symbolen

VRP Vehicle Routing Problem
MIT CTL Center for Transportation and Logistics van het Massachusetts Institute of Technology
(A)TSP Asymmetric Travelling Salesman Problem
X,Y zone, stop of halte
c kost van een transitie tussen twee zones, stops of haltes
i, j index van een zone, stop of halte in een volgorde of index van een beperking
n,k totaal aantal zones, stops of haltes
OX Ordered Crossover
PMX Partially Mapped Crossover
ACO Ant Colony Optimization
p penalty bij een bepaalde beperking of gemiddelde bezorgtijden voor pakketten
f(x) beperking geëvalueerd op volgorde x
SD Sequence Deviation
ERP Edit Distance with Real Penalty
hist waargenomen stop volgorde
sub gegenereerde stop volgorde
L-M.PR representatie van de symbolen van een zone ID
ord(.) ASCII waarde functie
v gemiddeld volume van pakketten
t reistijd voor een transitie tussen twee zones, stops of haltes
T een tour
A,B,C,D,E,F verschillende zones
$\Pr(i \to j)$ transitie kans tussen zones met matrix indices i en j
$\Pr(i \leftrightarrow j)$ connectie kans tussen zones met matrix indices i en j
w het gewicht van een beperking in de tweede stap
CE Categorical Cross-Entropy loss
AD Arc Difference
(D)NN (Deep) Neural Network
ASU-DNN Alternative Specific Utility DNN
LSTM Long Short-Term Memory
RNN Recurrent Neural Network
MLR Maximum Likelihood Routing
SOP Structured Output Prediction
Q, K-V Query, Key-Value
RL Reinforcement Learning
EM Expectation-maximization
GNN Generative Neural Network

Hoofdstuk 1: Introductie

1.1 Het belang van e-commerce en last-mile bezorgingen

De afgelopen jaren heeft e-commerce een exponentiele groei doorgemaakt. Het is vooral gedreven door het toenemend gebruik van het internet in het dagelijkse leven. Daarnaast heeft de COVID-19 pandemie de overgang naar e-commerce versneld. Volgens een onderzoek van Statista (2022) zal de verwachte omzet in de e-commerce markt in 2023 $4.11 miljard bedragen en met een jaarlijkse groei van 11.51% bedraagt het geprojecteerde markt volume voor 2027 $6.35 biljoen.

Door de groei van e-commerce wordt de relevantie van zijn componenten ook groter. Volgens het Capgemini Research Institute zijn last-milebezorgingen verantwoordelijk voor 41% van de totale logistische kost en is hierbij veruit de grootste factor (Jacobs et al., 2019).

In zijn simpelste vorm stelt een last-milebezorging een vehicle routing problem (VRP) voor waarbij verschillende bezorgers meerdere bezorgingen moeten maken en er een rit voor elke bezorger opgesteld moet worden. Er bestaan vele varianten van dit probleem, recent opkomende voorbeelden hiervan zijn ‘same-day’ en ‘on-demand’ bezorgingen die het probleem complexer maken.

1.2 De 2021 Amazon Last Mile Routing Challenge

Op 15 maart 2021 begon de 2021 Amazon Last Mile Routing Challenge. Deze onderzoeksuitdaging werd georganiseerd door het Last Mile research team van Amazon.com en werd ondersteund door het Center for Transportation and Logistics van het Massachusetts Institute of Technology (verder naar verwezen als MIT CTL). MIT CTL publiceerde de beste inzendingen en er werd een totaal bedrag van $175,000 gealloceerd aan prijzen voor de drie beste inzendingen. De deadline voor de inzendingen was 18 juni 2021. Deelnemers kregen dus ongeveer vier maand om aan hun onderzoek te werken. Er hebben in totaal meer dan 220 teams meegedaan (Gantenbein, 2023) en er zijn 45 inzendingen voor de finale ronde ingestuurd (MIT, z.d.-b).

Het onderzoek is gebaseerd op het verschil tussen theoretische route planning en werkelijke route uitvoeringen voor bezorgingen. Route optimalisatie problemen van

realistische groottes zijn moeilijk op te lossen. In de literatuur over operationeel onderzoek is er al veel onderzoek gedaan naar mogelijke oplossingen voor het VRP en zijn varianten (waaronder het TSP) in termen van kwaliteit van het resultaat en rekenkundige kost, startend met de klassieke paper Dantzig en Ramser (1959). In werkelijke operaties is de kwaliteit van een route niet enkel gedefinieerd door haar theoretische lengte, tijdsduur of kost. Dit verschil is te wijten aan de ontastbare kennis van ervaren bezorgers. Deze kennis kan variëren van aspecten over de geografische ligging van individuele bezorgingen tot aspecten over de infrastructuur zoals wegen, parkeerplaatsen of tijden wanneer er veel verkeer is. Deze aspecten worden vaak niet gemodelleerd in optimalisaties waardoor de routes die bezorgers maken variëren van de initieel geplande routes. Deze variaties kunnen mogelijks efficiënter zijn als de ontastbare kennis mee in het model zou worden gerekend. Het doel van de challenge is om gebruik te maken van artificiële intelligentie, machine learning, deep learning en andere niet-conventionele methoden om betere routes te produceren dan traditionele methodes in termen van kwaliteit en rekenkost (MIT, z.d.-a).

“Drivers, however, frequently deviate from those computed routes. Drivers carry information about which roads are hard to navigate, when traffic is bad, when and where they can easily find parking, which stops can be conveniently served together, and many other factors that existing optimization models don’t capture.” (Hardesty, 2022)

Op Figuur 1 wordt links de route getoond dat de optimalisatie software van het Last Mile team voorstelde aan een bezorger en rechts wordt de route getoond dat de bezorger koos om uit te voeren. Punten A en B zijn waar de routes starten (groen) en eindigen (zwart).

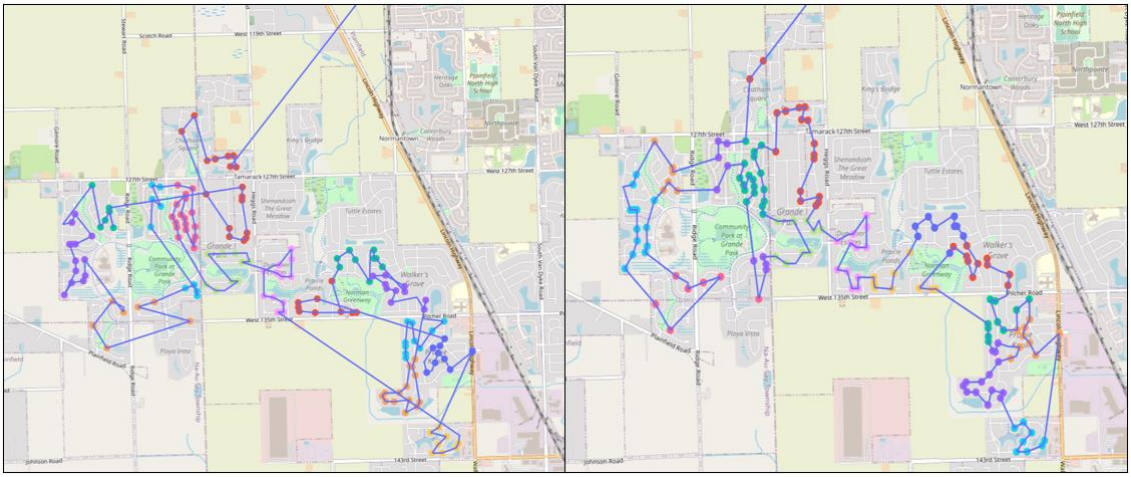


Figuur 1 : Voorgestelde route VS bezorger

De dataset gebruikt voor deze challenge bestaat uit 9,184 historische routes die uitgevoerd zijn geweest door Amazon bezorgers in 2018. Er zijn 6,112 routes voorzien voor de deelnemers zodat ze hun modellen hierop zouden kunnen baseren of trainen. Een kopie van deze training set is momenteel publiekelijk verkrijgbaar (MIT, 2022). Deze routes zijn verspreid over vijf verschillende grote steden in de VS: Los Angeles, Seattle, Austin, Chicago en Boston. De resterende 3,072 routes worden als verborgen test set gebruikt om de finale score van inzendingen te berekenen. Elke route is gelabeld met een categorische kwalitatieve score. Deze score kan ‘low’, ‘medium’ of ‘high’ zijn en kan gebruikt worden voor het tunen van een model tijdens training. De

verborgen test set bestaat alleen maar uit routes die een hoge kwaliteit hebben (Merchán et al., 2022).

Het verschil tussen een tour van een bezorger en een geoptimaliseerde tour wordt geïllustreerd aan de hand van de tweede route uit de dataset. Op Figuur 2 kan het verschil waargenomen worden tussen een route dat afgelegd is door een bezorger (links) en een route die geminimaliseerd is in functie van de totale afstand. De stops worden voorgesteld als gekleurde punten. De kleuren stellen verschillende zones voor. Hoewel de route van de bezorger chaotisch lijkt zijn er toch nog steeds veel gelijkenissen tussen de twee.

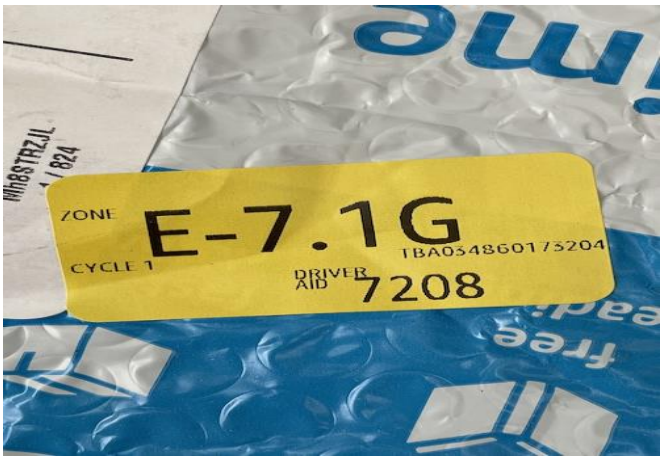


Figuur 2 : Voorbeeldroute bezorger (links) VS Google OR Tools (rechts)

Als de routes van de dataset gevisualiseerd worden wordt het meteen duidelijk dat chauffeurs een preferentie vertonen voor het afwerken van alle stops binnen een bepaalde zone vooraleer ze aan stops van een andere zone beginnen. Volgens de University of Waterloo (2022a) wordt dit gedrag veroorzaakt doordat de camionetten opgeladen worden met een zak per zone, waarbij elke zak de pakketjes bevat van de stops van die zone. Eens een zak leeg is kan ze opgeplooid en weg gestoken worden, zo maakt de chauffeur meer ruimte in de camionet en kan hij eenvoudiger toekomstige pakketjes uit andere zakken halen. Het is dan ook handiger om zakken van zones die eerder worden bezocht meer achteraan in de camionet de plaatsen zodat de bezorger er gemakkelijker aankan. Om deze reden speelt route planning ook al een rol bij het opvullen van de camionet. Op Figuur 3 wordt getoond hoe de zakken die de pakketjes per zone bevatten geplaatst worden in een camionet terwijl er op Figuur 4 een pakket getoond wordt waarop een gele sticker met de bijhorende zone ID plakt (Waterloo, 2022).

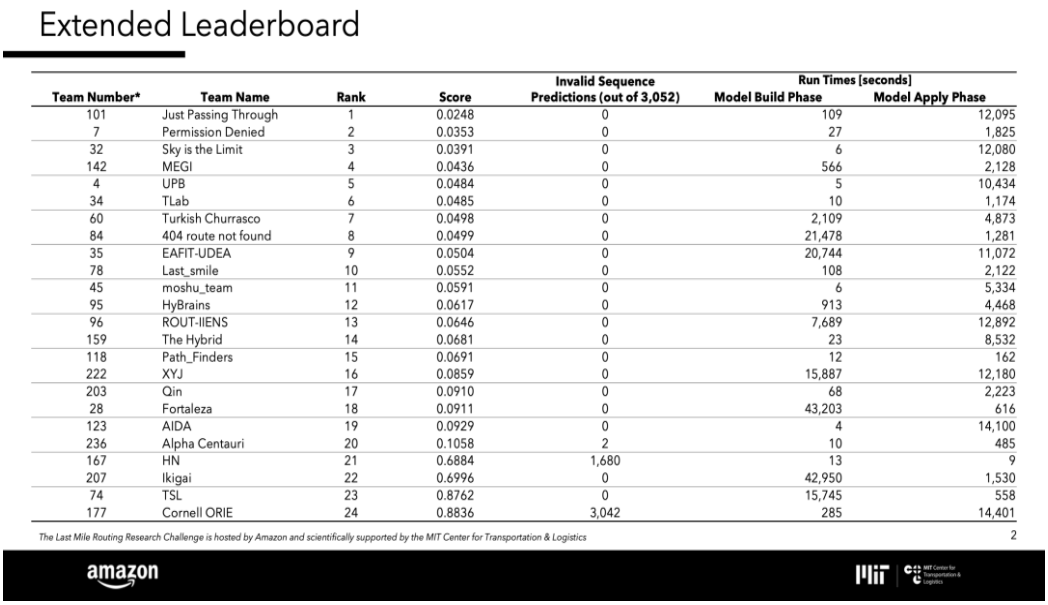


Figuur 3 : Zakken per zone in Amazon camionet



Figuur 4 : Pakket voor stop in zone E-7.1G

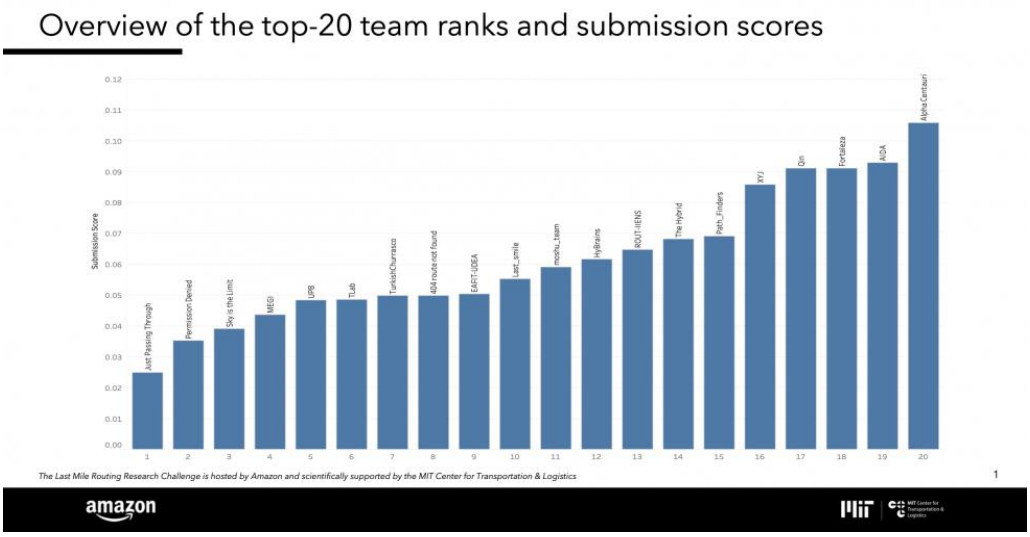
De kwaliteit van een voorgestelde route wordt gekwantificeerd op basis van de score functie van MIT-CAVE. Deze functie berekent de gelijkenis tussen de targetstops volgorde en de voorgestelde volgorde. De reistijden tussen de stops moeten ook worden meegegeven, een genormaliseerde versie hiervan wordt namelijk gebruikt bij het berekenen van de gelijkenisscore. Als de volgordes overeen komen, wordt een perfecte score van 0 behaald terwijl als de voorgestelde volgorde willekeurig gegenereerd wordt dan zou de score tussen 0.8 en 1.2 moeten zijn. Deze scorefunctie wordt gebruikt op elke ‘entry’ van de verborgen dataset om een gemiddelde score voor een model te bekomen. Het doel is dus om een model op te stellen die een zo laag mogelijke gemiddelde score behaalt en niet een model dat telkens de snelst mogelijke stop volgorde (in termen van reistijden) genereert (MIT CAVE Lab, 2021). Op Figuur 5 en Figuur 6 worden de behaalde scores van de beste teams getoond (MIT, 2021).



Extended Leaderboard

Team Number*	Team Name	Rank	Score	Invalid Sequence Predictions (out of 3,052)	Run Times [seconds] Model Build Phase	Model Apply Phase
101	Just Passing Through	1	0.0248	0	109	12,095
7	Permission Denied	2	0.0353	0	27	1,825
32	Sky is the Limit	3	0.0391	0	6	12,080
142	MEGI	4	0.0436	0	566	2,128
4	UPB	5	0.0484	0	5	10,434
34	TLab	6	0.0485	0	10	1,174
60	Turkish Churrasco	7	0.0498	0	2,109	4,873
84	404 route not found	8	0.0499	0	21,478	1,281
35	EAFIT-UDEA	9	0.0504	0	20,744	11,072
78	Last_smile	10	0.0552	0	108	2,122
45	moshu_team	11	0.0591	0	6	5,334
95	HyBrains	12	0.0617	0	913	4,468
96	ROUT-IIENS	13	0.0646	0	7,689	12,892
159	The Hybrid	14	0.0681	0	23	8,532
118	Path_Finders	15	0.0691	0	12	162
222	XYJ	16	0.0859	0	15,887	12,180
203	Qin	17	0.0910	0	68	2,223
28	Fortaleza	18	0.0911	0	43,203	616
123	AIDA	19	0.0929	0	4	14,100
236	Alpha Centauri	20	0.1058	2	10	485
167	HN	21	0.6884	1,680	13	9
207	Ikigai	22	0.6996	0	42,950	1,530
74	TSL	23	0.8762	0	15,745	558
177	Cornell ORIE	24	0.8836	3,042	285	14,401

Figuur 5 : Gerangschikte inzendingen voor de routing challenge



Figuur 6 : Histogram beste inzendingen voor de routing challenge

1.3 Verdere hoofdstukken en onderzoeksvragen

Elke route in de dataset kan dus gezien worden als de oplossing van een TSP aangezien elke stop exact een keer moet voorkomen in de uiteindelijke volgorde en TSP solvers kunnen dit garanderen. TSP solvers liggen daarom als basis van veel modellen en vormen een goed startpunt. De moeilijkheid ligt in het integreren van AI technieken met TSP solvers. Een algemene manier om dit te doen is om de reistijden matrix aan te passen zodat er rekening wordt gehouden met een opgestelde set van

soft constraints. De moeilijkheid hierbij ligt bij het opstellen van de juiste beperkingen en het tunen van de penalty waarden gekoppeld aan de beperkingen. Dit is ook wat het winnende team heeft gedaan in Cook et al. (2022), ze maakten gebruik van een set van beperkingen die zich vooral focust op de transities van zones door gebruikt te maken van de verschillen in de zone ID's, die een vaste structuur hebben. Deze thesis zal voortgebouwd worden op basis van het model van Canoy et al. (2022). Dit model is gebaseerd op een twee-staps structuur: in de eerste stap wordt een zone volgorde opgesteld terwijl in de tweede stap die zone volgorde wordt gebruikt als beperkingen om penalty's te genereren om het op te lossen van de TSP. Deze aanpak is gebaseerd op de preferentie van chauffeurs om zone per zone te bezoeken. Dit laat het toe om beide stappen en dus problemen, apart aan te pakken. Deze aanpak zou daarom ook minder expressief kunnen zijn omdat het zich limiteert tot een soort beperkingen in de tweede stap, namelijk de zone volgorde. De zone volgorde is iets dat gedefinieerd moet worden en dat niet expliciet wordt meegegeven. Het is ook niet vanzelfsprekend dat **een zone volgorde beperking genoeg informatie bevat om in de tweede stap tot een goede stops volgorde te komen**. Dit mogelijk probleem zal verder onderzocht worden aan de hand van enkele theoretische experimenten.

Het probleem in de eerste stap is dus om een gepaste zone volgorde op te stellen. Dit stelt opnieuw een gelijkaardig probleem voor als met de stops, het verschil is dat de transitie kost tussen twee zones onbekend is. Deze kosten worden in Canoy et al. (2022) bekomen aan de hand van een transitie kansen matrix waarbij de kansen bekomen worden door een Markov model. Dit model wordt opgesteld aan de hand van Markov counting over het training set. Er zal eerst onderzocht worden of er geen toepasselijkere methodes zijn om **Markov counting toe te passen zodat subtielere patronen in de data ook gemodelleerd kunnen worden**.

Het probleem van het bepalen van de richting van een zone volgorde is een probleem dat vaak voorkomt bij de winnende teams. Eens een zone of stop volgorde gegenereerd is kan die zowel van links naar rechts als van rechts naar links geïnterpreteerd worden. Beide geven verschillende uitkomsten omdat richting van belang is. Het effect van een referentie mechanisme om de richting van een zone volgorde te bepalen zal experimenteel getest worden. Aangezien dit in een postprocessing stap gebeurd is het interessant om te zien of het mogelijk is om **de generatie van de zone volgorde en het bepalen van de richting onafhankelijk van elkaar kunnen gebeuren**.

In Canoy et al. (2022) wordt er gesuggereerd om de Markov transitie kansen beter te benaderen door gebruik te maken van neurale netwerken. Het doel is om binnen een bepaald tijdslimiet een neuraal netwerk op te stellen die transitie kansen tussen twee zones genereert op basis van de symbolen in hun zone ID's en de preferenties van de bezorgers. Dit zal getest worden door gebruik te maken van ASU-DNN's door de utiliteit van elke transitie apart te bepalen. Dit zal aan de hand van drie verschillende geprobeerd worden. Eerst door gebruik te maken van verschillende linear layers om de utiliteiten te bepalen, dan door de bestaande Markoviaanse modellen te generaliseren aan de hand van een encoder die voor elke transitie een utiliteit bepaald en tot slot door met een gelijkaardige encoder design utiliteiten te bepalen zodat de transitiekansen zoveel mogelijk overeenkomen met de echt waargenomen transities.

Hier wordt ook gekeken of dat er mogelijks meer gedetailleerde preferenties zijn dan de algemene zone ID beperkingen. De assumptie die hier gemaakt en getest wordt is of dat het mogelijk is om **de utiliteit van transities aan de hand van hun individuele attributen te bepalen**.

Hoofdstuk 2: Literatuurstudie

2.1 Oorsprong van het TSP en oplossingen

Het TSP is een van de bekendste combinatorische optimalisatie problemen. Het probleem komt op in een variëteit van contexten, van het boren van gaten in een PCB plaat tot sequentiële DNA problemen oplossen (University of Waterloo, z.d.). De meest bekende vorm van het probleem is een routing probleem. Gegeven een set van steden die een agent moet bezoeken, wat is dan de optimale volgorde om ze te bezoeken en terug te komen bij de startpositie? In de simpelste vorm van dit probleem wilt men de totale afgelegde afstand minimaliseren. Dit is dan een symmetrische TSP, omdat de afstand tussen twee haltes $dist(X,Y)$ gelijk is aan de afstand in de andere richting $dist(Y,X)$ en dit geldt dan voor alle stadsparen (X,Y). Er bestaan ook andere vormen die asymmetrisch zijn, zoals wanner men de totale tijdsduur wilt minimaliseren en dus $time(X,Y) = time(Y,X)$ niet voor alle (X,Y) paren geldt. Er kunnen ook complexere objectief functies geminimaliseerd worden die bijvoorbeeld rekening houden met meerdere individuele kost functies.

Een formele wiskundige definitie van het probleem volgens Wikipedia contributors (2023) is als volgt:

$$\min \sum_{i=1}^{n} \sum_{j \neq i, j=1}^{n} c_{ij} x_{ij}$$

$$\forall j: \sum_{i=1, i \neq j}^{n} x_{ij} = 1$$

$$\forall i: \sum_{j=1, j \neq i}^{n} x_{ij} = 1$$

$$x_{ij} = \begin{cases} 1, & pad\ gaat\ van\ i\ naar\ j \\ 0, & anders \end{cases}$$

Op de eerste regel staat de objectief functie die in dit geval geminimaliseerd moet worden, waarbij c_{ij} de kost voorstelt voor de transitie boog tussen stad i en stad j. Hier wordt de toegekende kost van alle n transities opgesomd. De tweede en derde regel zorgen ervoor dat er telkens maar een transitie kan gedaan worden vanuit en naar een bepaalde stad. De laatste regel is wat het een integer programming probleem maakt. Deze regel zorgt ervoor dat een transitie ofwel gekozen wordt ofwel niet gekozen wordt, wat dus x_{ij} een binaire variabele maakt. Het probleem wordt als een

integer linear programming probleem beschouwd aangezien elke expressie lineair is maar er binaire variabele zijn.

De kostfunctie wordt dan gerepresenteerd aan de hand van een kostenmatrix C met dimensies $n \times n$. In deze matrix bevinden zich dan alle c_{ij} voor alle mogelijke combinaties van i en j. Een symmetrische kostfunctie heeft dan ook een symmetrische kostenmatrix C en alle elementen van C worden gegenereerd door deze kostenfunctie. Voor een asymmetrische kostenfunctie is de kostenmatrix ook asymmetrisch.

Er bestaan veel methoden om oplossingen van TSP's te zoeken. De meest voor de hand liggende is een brute force methode. Hierbij worden alle mogelijke oplossingen afgegaan die voldoen aan de beperkingen. Voor elke mogelijkheid wordt de finale kost berekend en als die lager is dan de huidige minimale kost dan wordt die de nieuwe minimale kost. Deze methode garandeert een exacte oplossing te vinden maar is niet schaalbaar. Het heeft namelijk een complexiteit van $O(n!)$ met n als aantal steden (Cormen et al., 2009). Een ander naïef algoritme is het greedy algoritme. Hierbij wordt telkens de dichtstbijzijnde stad gekozen, die nog niet gekozen is geweest, om een connectie te maken. Dit algoritme heeft een complexiteit van $O(n^2)$ maar de voorgestelde oplossingen zijn zelden optimaal (Kleinberg & Tardos, 2005). Een van de betere exacte algoritmes is het algoritme van Christofides. Voor dit algoritme moet het probleem ook voldoen aan de driehoeksongelijkheid $\forall X, Y, Z: dist(X, Y) < dist(X, Z) + dist(Z, Y)$. Het maakt gebruik van de minimum spanning tree van de graaf en het vinden van de minimale perfect matching van de steden met een oneven rank (een oneven aantal mogelijke verbindingen). De uitkomsten worden dan gecombineerd om een toer te genereren (Christofides, 2022).

Andere aanpakken zijn gebaseerd op linear programming technieken zoals de simplex methode, die een complexiteit van $O(2n)$ heeft. Deze kan gebruikt worden om het probleem exact op te lossen zonder de beperking dat de variabelen binair moeten zijn. Vervolgens wordt er rond de exacte oplossing naar een oplossing gezocht die wel voldoet aan de binaire restricties. Deze oplossing is weer niet noodzakelijk de beste (Dantzig, 1963). Het zoeken naar de exacte oplossing die wel aan de binaire restricties voldoet is niet eenvoudig, dit kan aangepakt worden met branch and cut methodes. Het principe erachter is om zoekruimtes af te sluiten waarvan het zeker is dat oplossingen daarbinnen slechter zijn. Deze methodes hebben weer een complexiteit van $O(n!)$ in het slechtste geval (Padberg & Rinaldi, 1991).

Er bestaan ook heuristieken dat gebaseerd zijn op het verfijnen van een oplossing of populatie van oplossingen. Vele zijn gebaseerd op de Lin-Kerninghan Heuristiek (LKH) . Deze heuristiek vertrekt vanuit een individu, een mogelijke oplossing, en past 2-opt of 3-opt moves toe op dat individu. 2- en 3-opt moves zijn local search methodes om een gegeven individu mogelijks te verbeteren. Bij een 2-opt herschikking worden twee verbindingen doorbroken en terug gemaakt op de enige andere manier zonder de toer te verbreken. Bij een 3-opt herschikking wordt hetzelfde gedaan maar met drie connecties, er zijn dan ook meerdere manieren om de toer terug te verbinden en leidt tot meer computaties. In beide gevallen is het does om een herschikking te vinden die minder kost dan ervoor. Hogere n-opt moves kunnen ook gebruikt worden en zullen leiden naar betere oplossingen maar zijn computationeel meer kostbaar. Als een

individu n-opt optimaal is dan wilt dit zeggen dat er geen herschikking van n connecties meer is dat de tour kan verbeteren (Lin & Kernighan, 1973).

Deze heuristiek wordt vaak gecombineerd met andere methoden om te voorkomen dat het vastloopt in een lokale minima. Simulated annealing kan hiervoor gebruikt worden. Het idee erachter is dat het iteratief algoritme dat het huidig individu telkens probeert te verbeteren gecontroleerd wordt aan de hand van een temperatuur functie. Deze temperatuur functie dicteert een kans die afhankelijk is van tijd dat het algoritme aan het runnen is of nummer van iteraties dat het al gedaan heft. De kans stelt de kans voor dat een herschikking wordt toegepast op het individu dat het individu verslechtert. Hoeveel het individu verslechtert wordt ook meegerekend in de kans. De temperatuur functie is vaak een exponentieel verval zodat deze kans in het begin van het algoritme relatief hoog is en op het einde heel laag wordt. Dit mechanisme maakt het mogelijk om uit lokale minima te geraken en mogelijks betere oplossingen te vinden (Kirkpatrick et al., 1983).

Een andere aanpak die gebruikt kan worden om uit lokale minima te geraken is aan de hand van een genetisch algoritme. Een genetisch algoritme start met een initiële populatie van individuen. Deze populatie kan gedigitaliseerd worden door gebruik te maken van andere goede en snelle maar niet optimale heuristieken (zoals bijvoorbeeld een stochastische heuristiek gebaseerd op de inverse kost van elke connectie) (Holland, 1992). Vervolgens worden goede individuen gerecombineerd met elkaar. In deze fase worden individuen door een selectie mechanisme (stochastisch of deterministisch) geselecteerd en gerecombineerd. De recombinatie gebeurd ook op basis van een vooraf gedefinieerd mechanisme. Enkele veelgebruikte mechanismen zijn bijvoorbeeld Ordered Crossover (OX) of Partially Mapped Crossover (PMX). Het doel van beide is om een nieuw kind individu te creëren vertrekkend vanuit twee ouder individuen die eerder geselecteerd werden. Het idee is dat het nieuw gecreëerd individu dan de goede karakteristieken van beide ouders zou moeten overnemen. Daarna worden enkele nieuwe individuen gemuteerd om voor meer variëteit en exploratie te zorgen. Tenslotte worden er individuen geëlimineerd, aan de hand van een eliminatie mechanisme, zodat de populatie constant blijft. Deze loop wordt dan opnieuw uitgevoerd tot er een bepaald stopping criterium wordt voldoet (zoals een maximaal aantal iteraties of tijd of een convergentie van de populatie). Andere mechanismen kunnen ook toegevoegd worden aan deze basis loop bijvoorbeeld om meer diversiteit te promoveren. Het doel van het algoritme is om met een goede balans tussen exploratie en exploitatie een goede oplossing te bereiken.

Enkele voordelen van local search methoden, simulated annealing en genetische algoritmen zijn dat ze met elkaar gecombineerd kunnen worden om een betere balans tussen exploratie en exploitatie te vinden. Een ander voordeel is hoe ze omgaan met restricties. Restricties hoeven geen formele wiskundige definitie te hebben om gerespecteerd te worden door deze algoritmen. De algoritmen gebruikten namelijk een fitness functie om individuele oplossingen te scoren. In het geval van een TSP is deze functie meestal de afstandenmatrix. De afstanden die afgelegd moeten worden om door de hele voorgestelde tour te gaan worden dan opgeteld. In deze functie kunnen ook andere restricties in verwerkt worden als termen van de vorm $p_i f_i(x)$. Hierbij is p_i

de penalty-waarde dat hoort bij de i-de beperking en $f_i(x)$ is de functie dat nakijkt of de voorgestelde oplossing x, die beperking respecteert of violeert. Als de beperking gerespecteerd wordt zou de uitkomst van $f_i(x)$ een 0 moeten zijn, dan wordt de bijhorende penalty-waarde p_i niet toegepast op de fitness functie. Als de beperking niet gerespecteerd wordt zou de uitkomst van $f_i(x)$ een 1 moeten zijn zodat de bijhorende penalty wel wordt meegerekend in de bij het scoren van dat individu. Strikte beperkingen kunnen dan gecodeerd worden voorgesteld door een heel grote waarde voor p_i te kiezen ten opzichte van de afstandenmatrix (bijvoorbeeld in et geval dat een deelrij van steden van een tour niet gedaan mag worden). Voor andere zachtere restricties moeten de penalty-waarden accurater gekozen worden. Het voordeel hiervan is dat het zoek algoritme zichzelf doorheen deze restricties navigeert en individuen die ze niet respecteren zal verwijderen omdat ze een slechtere kostscore gaan hebben. Een nadeel is dat het niet gegarandeerd is dat de finale voorgestelde oplossing aan alle strikte beperkingen zal voldoen. Hiervoor zijn onafhankelijke checks nodig op het einde van het zoekalgoritme.

Een laatste aanpak is de Ant Colony Optimization (ACO). Dit algoritme is gebaseerd op hoe mieren feromonen achterlaten op wegen die ze nemen. Het idee is dat als er veel feromonen op een weg liggen, dan is de kans groot dat het een belangrijke weg zal zijn (Dorigo & Stützle, 2010). Deze methode maakt gebruik van een kansenmatrix die kansen bevat voor elk mogelijke transitie. Deze kansen worden berekend op basis van a priori kennis (vaak is dit de inverse van de kostfunctie) en de feromonen. Telkens als er een tour gekozen wordt dan worden de feromonen voor alle transities aangepast afhankelijk van hoe goed de tour het doet. Wegen die vaak aanwezig zijn in de tour zullen dan ook een hogere feromonen parameter hebben en zullen een grotere kans hebben om nog eens geselecteerd te worden. Deze methode garandeert opnieuw geen exacte oplossing maar is wel heel robuust en schaalt goed voor problemen met een toenemend aantal steden. Een ander voordeel is dat het kan omgaan met problemen die dynamische aspecten hebben en mogelijks doorheen de tijd veranderen.

2.2 Algemene informatie over de challenge en dataset

Het doel van de challenge is om kennis over de complexe operationele omgeving uit de data af te leiden om veiligere en meer duurzame bezorgingen te kunnen doen. Voorbeelden hiervan zijn dynamische verkeerscondities, veilige en gemakkelijke parkeer mogelijkheden of de beschikbaarheid van klanten om de bezorging succesvol te kunnen afgeven. Deelnemers moeten dus een model construeren die gebruik maakt van historische waargenomen routing volgorden van stops om hoge kwaliteit route volgorden te kunnen voorstellen waarvoor de optimale routing nog niet gekend is. De creatie van het model wordt gebaseerd op historische data van ‘high’, ‘medium’ en ‘low’ kwaliteit routing volgorden.

De historische data bevat informatie over de route, de stops die geserveerd moeten worden, de pakketten, afstanden, de waargenomen route volgorden en de kwaliteit ervan. Het model kan dan als inputs informatie nemen over de routes, de stops, de

pakketjes en de afstanden. De output moet een routing volgorde zijn. De evaluatie van de voorgestelde output wordt dan gedaan door de proximiteit te berekenen tussen de voorgestelde route en de waargenomen route voor de nieuwe input data.

De structuur van de route informatie is als volgt:

- Route ID: een unieke ID voor een groep van stops die geserveerd moet worden en samen een route moeten vormen
- Station Code: een unieke ID dat de bezorgingen station voorstelt waarvan de bezorger de route start en eindigt
- Date: datum van wanneer de route afgelegd is geweest
- Departure Time: tijdstip van wanneer de bezorger de route gestart is
- Executor Capacity: capaciteit van de wagen waarmee de bezorger rijdt (in volume eenheid cm^3)
- Stops: de set van stops die geserveerd moeten worden door de bezorger
- Observed sequence: de waargenomen volgorde van stops die de bezorger heeft geserveerd
- Route Type: categorische waarde die de kwaliteit voorstelt van de waargenomen volgorde (‘high’, ‘medium’, ‘low’)

De structuur van de stop informatie is als volgt:

- Stop ID: een unieke ID die elke stop van een route voorstelt (is uniek binnenin een specifieke route)
- Latitude en Longitude: werkelijke coördinaten van elke stop (worden licht verstoord voor privacy redenen)
- Type: categorische waarde die voorstelt of de huidige stop een station of een te serveren stop is (‘station’, ‘stop’)
- Zone ID: unieke ID van de geografische planning area waarin de stop zich bevindt
- Packages: de pakketten die bij elke stop gebracht moeten worden
- Distances: reistijden naar elke andere stop binnenin dezelfde route

De structuur van de informatie over pakketten is als volgt:

- Package ID: een unieke ID voor elk pakket (uniek doorheen heel het dataset)
- Status: categorische waarde die de staat van een pakket voorstelt (‘DELIVERED’, ‘DELIVERY_ATTEMPTED’, ‘REJECTED’)
- Time window: start en eind tijdstippen van wanneer bepaalde pakketten bezorgd kunnen worden
- Planned service time: verwachte tijdsduur dat het serveren van het pakket zal innemen
- Dimensions: maximale breedte, lengte en hoogte van het pakket (in cm)

Al deze data wordt voorzien voor 6,112 routes in een set van JSON files (MIT Center for Transportation & Logistics, 2021).

De evaluatie functie combineert de voorgestelde volgorde en de werkelijke volgorde om een gelijkenis score te geven. Het combineert elementen uit de Sequence Deviation en de Edit Distance with Real Penalty. De functie is gedefinieerd als volgt:

$$score = \frac{SD(hist, sub)ERP_{norm}(hist, sub)}{ERP_e(hist, sub)}$$

Waarbij $hist$ de waargenomen volgorde van stops is en sub de ingezonden volgorde is. $SD(hist, sub)$ stelt de Sequence Deviation voor van sub ten opzichte van $hist$, $ERP_{norm}(hist, sub)$ stelt de Edit Distance with Real Penalty voor toegepast op $hist$ en sub waarbij de afstanden worden genormaliseerd en $ERP_e(hist, sub)$ stelt het aantal edits voor dat berekend wordt door het ERP algoritme op de sub ten opzichte van de $hist$. $SD(hist, sub)$ is een maat van hoe verschillend de twee volgordes van elkaar zijn en neemt een waarde aan tussen 0 (als de volgorden gelijk zijn aan elkaar) en 1. Het meet alleen verschillen in ordering van de stops en is dus onafhankelijk van de afstanden tussen de stops. Het ERP algoritme is gebaseerd op de Levenshtein afstand. Het meet hoeveel single-element operaties (inserties, verwijderingen en substituties) er nodig zijn om sub te transformeren tot $hist$. Deze operaties worden gewogen met de afstanden tussen de stops die betrokken zijn bij de operaties. Als $ERP_e(hist, sub)$ gelijk is aan 0 dan wordt de uitkomst vervangen door 0. In het geval dat $hist$ gelijk is aan sub dan is de score gelijk aan 0. Als sub willekeurig gegenereerd wordt dan bevindt de score zich normaal tussen 0.8 en 1.2. De score van een model wordt dan berekend door de scores te berekenen op alle 3,072 routes uit de verborgen test set en het gemiddelde ervan te nemen. De afstanden die gebruikt worden bij het ERP algoritme zijn de reistijden tussen de stops. Ongeldige volgordes worden gepenaliseerd door ze te veranderen door willekeurige volgordes (MIT CAVE Lab, 2021)

2.3 Eerdere aanpakken en dataset analyses

De methodologieën en analyses van de drie winnende teams worden in deze sectie besproken om vaak voorkomende problemen en methoden te detecteren. Uitleg over de technische aanpak van alle deelnemers kan teruggevonden worden in de Technical Proceedings van Winkenbach (2021). Indien er geen andere bron vermeld staat bij de komende subsecties, wordt er gerefereerd naar de Technical Proceedings.

2.6.1 Sky is the Limit

Op de derde plaats stonden Okan Arslan en Rasit Abay met een eindscore van 0.0391. In een initiële descriptieve analyse maken ze de volgende waarnemingen over de data:

- 4.78% van de stops hebben tijdsintervallen waarin de pakketten bezorgd zouden moeten worden. Deze kunnen een impact hebben op de routes
- Reistijden zijn een van de belangrijkste features
- In de waargenomen routes zijn de zone ID's van twee opeenvolgende stops gelijk in 85.2% van de gevallen

- In de waargenomen routes veranderen alleen de 3de of 4de symbolen van de zone ID's van twee opeenvolgende stops in 11.3% van de gevallen
- In de waargenomen routes zijn de eerste en de laatste stops niet perse dichtbij het depot

Het impact van tijdsintervallen voor de pakketten wordt verder bestudeerd aangezien het een invloed kan hebben op welke TSP solver er gebruikt zal moeten worden. In de hele dataset worden er 1,457,175 pakketten bezorgd over de 6,112 routes. Van deze pakketten hebben er 113,993 een tijdsinterval beperking, wat 7.82% vormt van het totaal aantal pakketten. Deze observatie wordt gecombineerd met het feit dat de routes in de test set geen bezorgingen opnieuw geprobeerd zullen worden en met het feit dat de laatste levertijd van een pakket bij een stop altijd later is dan de vroegste levertijd bij diezelfde stop. De conclusie hieruit is dat het suboptimaal is om een stop meerdere keren te bezoeken en dat alle pakketten in een bezoek geleverd zouden moeten worden. Dus worden de tijdsintervallen niet per pakket beschouwd maar per stop. Van de 898,415 totale stops die bezocht worden hebben er 59,172 tijdsbeperkingen wat 6.58% voorstelt. Maar de vroegste tijdsbeperking van 32,711 van deze stops wordt automatisch voldaan (door te kijken wanneer de bezorger vertrekt en hoelang het minstens duurt om van het station naar de stop te rijden) en de laatste tijdsbeperking van 28,277 stops wordt ook automatisch gerespecteerd omdat de tijd waarop de laatste bezorging bij de laatste stop volbracht wordt nog binnen het tijdsinterval is van de tijdsbeperkingen. Als de intersectie van deze stops wordt genomen en wordt verwijderd uit de initiële 59,172 stops dan blijven er 42,989 stops over die 4.78% voorstellen.

Wanneer de routes worden opgelost met een TSP solver op basis van de reistijden wordt er een eind score van 0.0795 behaald . Dit impliceert dat bezorgers snellere routes prefereren.

In de 6,112 routes zijn er gemiddeld 21.04 verschillende zone ID, met de station meegerekend, per route en gemiddeld worden er 23.61 zone transities gedaan in een route. Dit wijst erop dat bezorgers liever alle stops binnen in een zone bezoeken vooraleer ze stops van een volgende zone gaan bezoeken. Zone ID's hebben de volgende structuur $L-M.PR$ waarbij L en R letters zijn en M en P getallen zijn. Twee opeenvolgende stops bevinden zich 85.2% van de gevallen in dezelfde zone. Een verschil functie tussen twee zone ID's wordt gedefinieerd door de vier symbolen van beide ID's af te gaan en als de corresponderende symbolen verschillend zijn wordt de afstand met 1 vergroot. De functie stelt dus voor hoeveel symbolen ze verschillend hebben. 11.9% van de transities tussen stops gebeuren tussen stops die een zone verschil hebben van een symbool. Van dit 11.9% zijn 11.3% transities waarbij alleen het 3de of het 4de symbool veranderd.

De gemiddelde tijdsduur om van het station naar de eerste stop te gaan en om van de laatste stop terug naar het station te gaan is 29% van de volledige tijdsduur van de route. Alleen in 35% van de routes is de eerste stop in de eerste 10% van de dichtstbijzijnde stops aan het station. Voor de laatste stop is dit 23%. Deze percentages zijn relatief laag.

In hun methodologie wordt er besloten om de tijdsintervallen voor de pakketten te negeren aangezien de percentage van pakketten die een speciale behandeling daarvoor

nodig had vrij laag was. Uit testen bleek ook dat een TSP met time windows het eindresultaat niet verbeterde, computationeel duurder was en mogelijks geen oplossing kon vinden. Als TSP solver werd een genetisch algoritme gebruikt die gebruik maakt van edge assembly crossover operator, ontworpen door Nagata en Kobayashi (2013). Het netwerk wordt getransformeerd zodat er penalty's worden toegepast op de reistijden. Er worden penalty's gebruikt op basis van de symbolen van zone ID's bij zone transities en om het belang van de eerste en de laatste stop te controleren. Een vaste set van penalty's werd gebruikt, het algoritme heeft dus geen training fase nodig.

Voor de train/validatie split worden er 1107 hoge kwaliteit routes dat geen 'DELIVERY_ATTEMPTED' bevatten als validatie set gebruikt omdat deze het meest overeenkomen met die van de verborgen test set. Op hun validatie set behaalden ze dan een gemiddelde score van 0.0367.

2.6.2 Permission Denied

Op de tweede plaats stonden Xiaotong Guo, Baichuan Mo en Qingyi Wang met een eindscore van 0.0353. De analyse en methodologie worden besproken aan de hand van Guo et al. (2023) dat een meer uitgebreide uitleg bevat dan de Technical Proceedings van Winkenbach (2021).

Veel waarnemingen dat werden gemaakt werden al besproken in de vorige subsectie over het team op de derde plaats maar anderen zijn nieuw. Er werd waargenomen dat er 'major zones' en 'inner zones' bestaan. Deze major zones worden geïdentificeerd aan de hand van de eerste twee symbolen van een zone ID. Als een zone ID de structuur $L-M.PR$ heeft, dan is $L-M$ de major zone waarbij die zone hoort, PR is dan de inner zone. Bezorgers prefereren om eerst alle stops binnen dezelfde major zones te serveren vooraleer ze verder gaan naar de volgende major zone. Deze claim wordt ondersteunt doordat 87.67% van alle transities binnen dezelfde major zone gebeuren.

Een andere belangrijke waarneming is dat binnen dezelfde major zone 82.49% van de transities gebeuren tussen twee zones met een verschil van inner zones van exact één. Het verschil van één is hier anders gedefinieerd dan bij het vorige team. De definitie is als volgt:

$$\left|P_j - P_i\right| + \left|ord\left(R_j\right) - ord(R_i)\right| = 1$$

Deze definitie is strikter dan die van het vorig team. Hier stellen P_j en P_i de derde symbolen voor van de zones waartussen de transitie gebeurt en R_j en R_i de vierde symbolen van diezelfde zones. Aangezien R_j en R_i letters voorstellen wordt de ord erop toegepast zodat hun index in het alfabet wordt verkregen. Er mag dus niet alleen een symbool verschillend zijn maar het verschil mag ook niet groter zijn dan een.

Een gelijkaardige waarneming is dat wanneer er een transitie is tussen twee verschillende major zones dat deze waarneming dan ook geldt maar dan met het paar M en L in plaats van P en R. Er is dus een verschil van één ook bij major zone transities en deze gebeuren in 96.17% van de gevallen.

Ook dit team besloot om aanpassingen te maken aan de reistijden matrix. Een penalty word toegepast op de reistijden tussen het station en de stops van een zone als die zone niet in de h meest dichtbij zijnde zones bij het station is in termen van gemiddelde reistijd en Euclidische afstand. De nieuwe reistijd wordt dan $t_{0i}\alpha$. Hier is t_{0i} de initiële reistijd om van het station naar stop i te gaan, α en h zijn hyperparameters. Reistijden t_{ij} tussen stops van twee verschillende zones worden vermenigvuldigd met een hyperparameter β als ze zich niet in dezelfde major zone bevinden. Reistijden t_{ij} voor stops van twee verschillende zones worden vermenigvuldigd met een hyperparameter γ als ze niet voldoen aan de beperking van het verschil van één. Alle hyperparameters werden dan gezocht aan de hand van een grid search.

De reistijden matrix van de stops wordt vervolgens opgelost aan de hand van een ILP solver om een stop volgorde te genereren. Er werd gemerkt dan veel gegenereerde volgorden deels of volledig geïnverteerd waren (bijvoorbeeld $A \to B \to C \to D$ in plaats van $D \to C \to B \to A$). Om dit probleem aan te pakken wordt er gebruik gemaakt van enkele post-processing technieken.

De technieken zijn gebaseerd op de waarnemingen dat de meeste bezorgers prefereren om business zones eerst te serveren en dat ze prefereren om grotere pakketten eerst te leveren. De eerste waarneming komt mogelijks doordat pakketten in business zones langere levertijden hebben, door ze eerst te brengen hebben bezorgers meer flexibiliteit tegen het einde van de rit. De tweede zou te maken kunnen hebben met dat als grote pakketten eerst worden geleverd dat er dan sneller meer ruimte wordt vrijgemaakt in het vervoermiddel, wat andere bezorgingen vergemakkelijkt. Dit is ook meer brandstof efficiënt.

Beide waarnemingen worden gebruikt om voor elke gegenereerde route te kijken of het beter zou zijn om ze volledig te inverteren. Hiervoor wordt eerst gekeken of:

$$\frac{p_i^-}{p_i^+} \geq \theta$$

waarbij p_i^+ en p_i^- de gemiddelde bezorgtijden voor de pakketten voorstellen van de eerste en laatste $p\%$ stops in route i en θ is een hyperparameter. Als de vergelijking niet voldaan wordt dan wordt de hele volgorde geïnverteerd. In het ander geval wordt een tweede gelijkaardige vergelijking gebruikt:

$$\frac{v_i^-}{v_i^+} \geq \mu$$

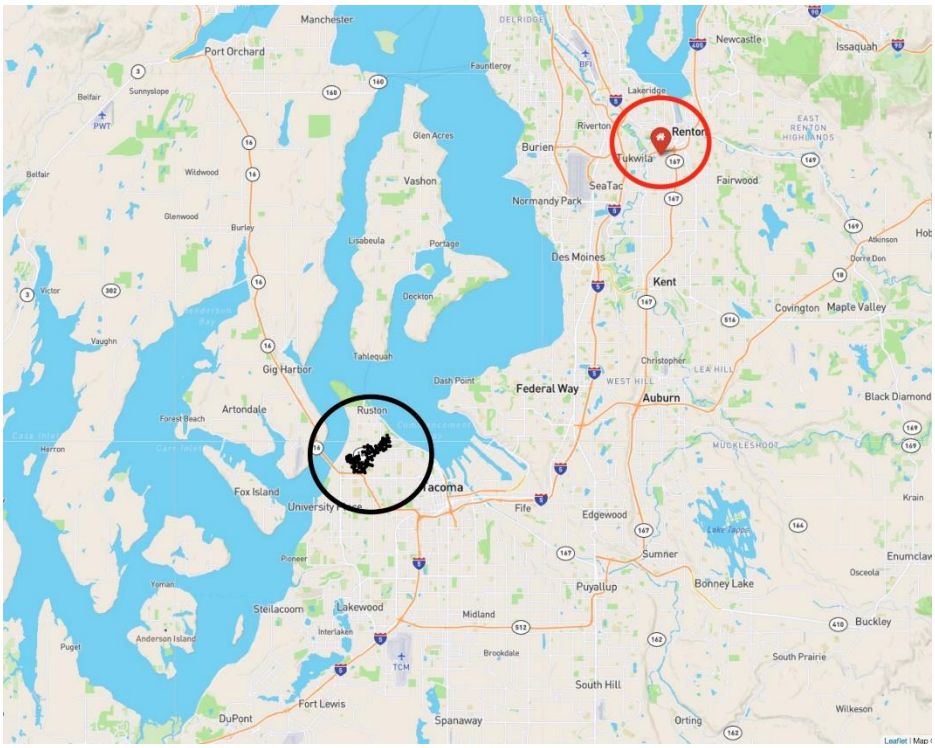
waarbij v_i^+ en v_i^- de gemiddelde volumes ($hoogte \times breedte \times diepte$) van de pakketten voorstellen van de eerste en de laatste $v\%$ stops in route i en μ is een hyperparameter. Opnieuw, als deze vergelijking niet wordt voldaan dan wordt de volgorde geïnverteerd en anders gebeurt er niets.

Op de gebruikte training (4889) en validatie (1223) split wordt ene score van 0.0381 verkregen. Er werd ook waargenomen dat indien de zone volgorde perfect gekend is maar de stops binnenin elke zone in een random volgorde gekozen worden dan word de score 0.0206. Zelfs na de postprocessing om de richting van de gegenereerde oplossing

beter te voorspellen is het probleem niet helemaal opgelost. De gebruikte postprocessing zorgt ervoor dat de score daalt van 0.0391 naar 0.0381 maar als inversies perfect gebruikt zouden worden zou de score 0.0280 zijn.

Een laatste opmerking dat wordt gemaakt is dat het interessant zou zijn om het probleem te versimpelen als volgt: aangezien de zone ID patronen zo sterk en bijna deterministisch zijn (bv. wanneer er een verschil van één is tussen twee zones op het derde symbool P, dan is er bijna zeker een connectie tussen die twee zones) is het probleem eerder welke zone de eerste of laatste moet zijn. Eens die met zekerheid gekend is dan is het vaak simpel om de resterende volgorde van zones te voorspellen. Er wordt voorgesteld om dit te proberen aan de hand van informatie over op- en afritten van snelwegen.

Een voorbeeld waar snelweg informatie handig kan zijn wordt geïllustreerd op Figuur 7 (University of Waterloo, 2022).



Figuur 7 : Station (rood) en cluster van stops (zwart) voor route#0002

2.6.3 Just Passing Through

Op de eerste plaats stonden William Cook, Stephan Held en Keld Helsgaun met een eindscore van 0.0248. De analyse en methodologie worden besproken aan de hand van Cook et al. (2021) en niet de Technical Proceedings van Winkenbach (2021).

Het team maakt gebruik van een local search gebaseerde TSP solver die gebruik maakt van het k-opt algoritme van Lin en Kernighan (1973). Het algoritme integreert beperkingen aan de hand van de $pen(T)$ functie die opereert op een tour T en in 0 resulteert als en slechts als tour T aan alle beperkingen voldoet. De functie $len(T)$ stelt de tijdsduur voor van tour T. In plaats van $pen(T)$ en $len(T)$ te combineren tot een functie worden ze apart gehouden. Een nieuwe tour $\hat{T}$ verbetert de oude tour T als en slechts als $pen(\hat{T}) < pen(T)$ en $len(\hat{T}) < len(T)$. Nieuwe tours worden gegenereerd aan de hand van k-opt moves.

Om gebruik te maken van algoritmen en optimalisaties die specifiek zijn voor symmetrische TSP wordt het ATSP symmetrisch gemaakt door ‘dummy’ stops toe te voegen. Elke stop wordt verdubbeld zodat er een inkomende dummy stop is en een uitgaande echte stop is. De afstand tussen de corresponderende inkomende en uitgaande stops is 0 en de verbinding tussen de twee moet verplicht aanwezig zijn in de oplossing. Door deze transformatie wordt een ATSP met n stops een TSP met $2n$ stops.

Voor het LKH algoritme worden alleen maar 3- en 4-opt moves gebruikt die deelrijen niet van richting inverteren. Dit is nodig omdat het anders problematisch zou zijn door de

transformatie van ATSP naar TSP. Voor dezelfde reden kunnen 2-opt moves niet gebruikt worden aangezien ze altijd een deelrij van richting inverteren.

Gedurende een tijdsperiode wordt de iterated local search meerdere keren uitgevoerd. Elke finale oplossing wordt geëvalueerd aan de hand van:

$$v(T) = PenaltyMultiplier \cdot pen(T) + len(T)$$

waarbij $PenaltyMultiplier$ een hyperparameter is. Het tour T met de laagste $v(T)$ waarde wordt dan als oplossing gehouden.

De gebruikte train/validatie split was hier ook met de 1107 hoge kwaliteit routes waarbij alles geleverd is als validatie set en de rest als training set. Door de 1107 validatie routes te genereren aan de hand van de solver zonder enige bijkomende beperkingen werd een score van 0.0703 verkregen. Als er een eenvoudige clustering beperking wordt gebruikt dan wordt de score 0.0486. Een simpele clustering beperking houdt in dat het veranderen van zone wordt gepenaliseerd door bij alle transities tussen twee stops uit verschillende zones een grote penalty constante bij op te tellen.

Beperkingen in verband met tijdsintervallen waarin pakketten geleverd moeten worden, zijn ook uitgetest. Dit is succesvol in de zin dat er tours geproduceerd worden die te tijdsintervallen beperkingen volledig respecteren maar ze halen een score van 0.04861, wat dus geen verbetering is. Voor deze reden worden deze beperkingen uit het model gelaten.

Er wordt waargenomen dat indien de perfecte sequentie van de zones gekend is , dat er dan een score van 0.0077 kan behaald worden door ze zo vaak mogelijk te respecteren. Als er ook nog rekening wordt gehouden met reistijden en de beperkingen er dus mee worden geïntegreerd, dan kan een score van 0.0050 behaald worden.

De volgende set van beperkingen dat wordt geïntroduceerd zijn voorrang beperkingen in verband met de zones. Hierbij wordt voor elke route uit de training set een gerichte graaf geconstrueerd van de zones. Een voorbeeld hiervan zou kunnen zijn:

$$A \rightarrow B \rightarrow C \ \rightarrow A \ \rightarrow D \rightarrow E \rightarrow F \ \rightarrow E$$

waarbij $\{A, B, C, D, E, F\}$ de set van zones voorstelt die bezocht zijn geweest. Merk op dat zones A en E meerdere keren worden bezocht door de bezorger. Deze graaf wordt dan opgesplitst in component paden (Tarjan, 1972). Het component pad voor de voorbeeld route wordt dan:

$$\{A, B, C\} \rightarrow \{D\} \rightarrow \{F, E\}$$

Deze structuur wordt gebruikt om voorrang beperkingen te genereren van de vorm:

$$index(X) < index(Y)$$

Waarbij X en Y twee zones uit twee verschillende componenten zijn en het component van X komt vlak voor het component van Y. Deze beperking houdt in dat X voor Y zou

bezocht moeten worden in een gegeven tour. Voor zones uit het zelfde component kunnen geen voorrangen bepaald worden. Dit is omdat ze cyclisch bezocht worden.

Dit mechanisme wordt gebruikt op nieuwe sets van zones door een referentie route te zoeken uit het training set die zoveel mogelijk gemeenschappelijke zones heeft met de set van zones. Voorrangsbeperkingen worden dan gegenereerd alleen voor de gemeenschappelijke zones. Componenten die geen gemeenschappelijke zones bevatten worden uit het componenten pad verwijderd. Voor een nieuwe set van zones $\{A, B, E, G\}$ zou het pad:$\{A, B, C\} \rightarrow \{F, E\}$ worden. Hieruit kan dan de volgende set van beperkingen gegenereerd worden:

$$\{index(A) < index(E), index(B) < index(E)\}$$

Er wordt ook getest door beperkingen te genereren aan de hand van de transitieve afsluiting van het eind pad.

> "The transitive closure of a graph G = (V,E) is a graph G* = (V,E*) such that E* contains an edge (u,v) if and only if G contains a path (of at least one edge) from u to v." (*Boost Graph Library: Transitive Closure - 1.55.0*, z.d.)

Dit houdt in dat er ook beperkingen gegenereerd worden voor zones die in componenten waren die niet noodzakelijk vlak voor of na elkaar komen maar meer algemeen voor of na elkaar komen.

De resultaten na het gebruik van deze beperkingen waren 0.0330 voor de beperkingen gegenereerd uit het componenten pad en 0.03169 voor componenten gegenereerd aan de hand van de transitieve afsluiting van het componenten pad. Er werd ook getest met meerdere referentie routes te nemen voor een nieuwe set van zones maar dat resulteerde in slechtere resultaten. De beperkingen werden samengevoegd met de beperkingen van de geclusterde ATSP aanpak.

Ook door dit team werden de patronen in de zone ID's gevonden. Er worden clusters van zone ID's gedefinieerd waarbij telkens een symbool wordt weggelaten. In totaal resulteert dit dan in vier sets van zone clusters (van klein naar groot):

$$\{L - M.PR\}, \{L - M.R\}, \{L - M\}, \{L\}$$

De splitsing is gemaakt geweest op basis van hoe vaak elk symbool verandert. P verandert het vaakst, dan R, M en L in deze volgorde. Multi-level cluster beperkingen worden dan toegevoegd zodat er telkens als een cluster verandert wordt in de volgorde dan wordt er een penalty bijgerekend. Dit leidt dan tot een score van 0.0271. Deze aanpak is vergelijkbaar met de 'major' en 'inner' zones van het vorig team dat al besproken is.

Verder wordt er ook waargenomen dat zones die achter elkaar komen en zich in dezelfde $\{L - M.R\}$ cluster bevinden een patroon volgen voor de P symbolen. Ze vormen namelijk een rij van getallen in toenemende of afnemende volgorde. Wanneer er een transitie gebeurd tussen zones uit verschillende $\{L - M.R\}$ clusters dan zijn de P symbolen vaak gelijk aan elkaar en verschillen de R symbolen van een. Hetzelfde principe geldt ook

voor clusters van hogere levels waarbij het laatst ontbrekend symbool een toenemende of afnemende volgorde vormt met de andere zones. Dit fenomeen wordt geïllustreerd op Figuur 8 (University of Waterloo, 2022).

amz0002	amz0003	amz0012	amz0019
Depot	Depot	Depot	Depot
A-2.2E	B-11.1G	A-2.1E	M-10.3D
A-2.1E	B-11.1H	A-2.2D	M-10.3C
A-2.1D	B-11.2H	A-2.3D	M-10.2C
A-2.2D	B-12.1G	A-2.3C	M-10.1C
A-2.3D	B-12.3G	A-2.2C	M-10.1B
A-2.3C	B-12.2G	A-2.1D	M-10.2B
A-2.2C	B-12.3G	A-2.1C	M-10.3B
A-2.1C	B-12.1H	A-2.1B	M-10.3A
A-2.1B	B-12.2H	A-3.2A	M-10.2A
A-2.2B	B-12.3H	A-2.1A	M-10.1A
	B-12.3J	A-2.2B	M-10.2A
	B-12.2J	A-2.3B	P-10.1A
	B-12.1J	A-2.3A	P-10.2A
	B-11.1J	A-3.1A	P-10.3A
	B-11.2J	A-2.2A	P-10.3B
	B-11.3J	A-2.3A	P-10.2B
	B-11.3H		P-10.1B
	B-11.3J		P-10.1C
	B-11.2G		P-10.2C
			P-10.3C
			P-10.3D

Figuur 8 : Volgorde van Zone ID's in waargenomen tours

Deze waarnemingen worden omgevormd tot gesorteerde zone beperkingen waarbij er penalty's worden toegepast op transities tussen zones die niet aaneenschakelende symbolen hebben. Hiermee werd een score van 0.0204 behaald. De gebruikte set van beperkingen is vergelijkbaar met de verschil van één beperkingen van het vorige team.

2.7 Maximum Likelihood Routing en het 2-staps algoritme

In deze sectie wordt in detail besproken hoe het tweestaps algoritme van Canoy et al. (2022) werkt en op basis van de principes uit Canoy en Guns (2019) waarop het geconstrueerd is. Op het einde is er een korte conclusie waarin de voor-en nadelen van deze aanpak en de resultaten worden besproken.

2.7.1 Het gebruik van een Markov model

Deze sectie wordt besproken aan de hand van het werk van Canoy en Guns (2019).

Het basis framewerk is gebaseerd op Canoy en Guns (2019). Het doel van het onderzoek in Canoy en Guns (2019) is gelijkaardig aan die van de routing challenge, namelijk om preferenties van chauffeurs te analyseren, begrijpen en kunnen gebruiken om betere routes voor te kunnen stellen zonder een exact optimalisatie probleem te

moeten definiëren. De moeilijkheid in het definiëren vaan een optimalisatie probleem is dat er vaak niet exact geweten is wat er geoptimaliseerd moet worden. Dit kan benaderd worden door data analyses te maken maar dat vraagt vaak veel werk en is geen algemene aanpak. Sommige patronen kunnen namelijk moeilijk zijn om te vinden. Bovendien zijn er niet noodzakelijk patronen in de data waaruit beperkingen kunnen gehaald worden om een nauwkeurigere optimalisatie functie te construeren.

De methode waarmee dit bereikt wilt worden is aan de hand van een eerste-orde Markov model. Het doel van het construeren van dit Markov model is zodat het gebruikt kan worden om de transitiekansen tussen de stops de berekenen. Deze transitiekansen kunnen dan gebruikt worden om het Maximum Likelihood Routing (MLR) probleem op te lossen. In een MLR probleem wordt de volgorde gezocht die de grootste kans heeft om voor te komen. Bij het gebruik van een eerste-orde Markov model is het MLR probleem als volgt gedefinieerd:

$$\max \prod_{(i,j)\in S} \Pr(volgende\ stop = j | huidige\ stop = i)$$

In deze functie zijn i en j opeenvolgende stops uit de geordende lijst S van alle stops die bezocht moeten worden. De kans term zal voor de eenvoud verder geschreven worden als $\Pr(i \rightarrow j)$ en beschrijft de kans dat de bezorger vanuit stop i naar stop j zal gaan en dus ook dat het pad x_{ij} tussen i en j in zijn tour zal zijn. Wat er gemaximaliseerd moet worden is in dit geval het product van de kansen van alle individuele transities die in de route aanwezig zijn.

Een kansenmatrix $T = [t_{ij}]$ met $t_{ij} = \Pr(i \rightarrow j)$ kan dan gebruikt worden om alle transitiekansen gestructureerd op te slagen. Het maximaliseren van het product van transitiekansen is equivalent aan het maximaliseren van de som van hun logaritmen:

$$max \sum_{(i,j)\in S} \log(\Pr(i \rightarrow j))$$

Dit kan ook veralgemeend worden door gebruik te maken van binaire beslissing variabelen:

$$max \sum_{(i,j)\in A} \log(\Pr(i \rightarrow j))\, x_{ij}$$

i en j zitten nu niet meer in S maar in A, wat de set van alle stops voorstelt. x_{ij} stalt de binaire variabele voor die 1 is als stop j vlak na stop i komt in de volgorde en 0 in elk ander geval is. Verder kan het probleem nog herschreven worden als:

$$min \sum_{(i,j)\in A} -\log(\Pr(i \rightarrow j))\, x_{ij}$$

Op deze manier moet de objectief functie geminimaliseerd worden. Zo kunnen CVRP of algemene TSP solvers gebruikt worden op de matrix $C = [c_{ij}]$ waarbij:

$$c_{ij} = -\log(t_{ij}) = -\log\left(\Pr(i \to j)\right).$$

Dit Markov model wordt opgesteld door te tellen hoe vaak een bepaalde transitie gekozen wordt. Deze hoeveelheden worden bijgehouden in een tel matrix. Deze matrix wordt gebruikt wanneer een nieuwe route voorspeld moet worden. De tellen van alle transities dat relevant zijn voor de nieuwe route worden gebruikt om een nieuwe submatrix te construeren. De rijen van nieuwe route-specifieke submatrix worden vervolgens gesmooth (aan de hand van een Laplace smoothing bijvoorbeeld) en genormaliseerd. Op deze manier wordt een matrix verkregen die op elke rij een kansendistributie heeft waarbij een kans geattribueerd wordt aan elke mogelijke transitie binnen een rij. Uit de conditionele kansen wordt het volgende verkregen:

$$\Pr(volgende\ stop = j \mid huidige\ stop = i) = \frac{\Pr\left(volgende\ stop = j, huidige\ stop = i\right)}{\Pr\left(huidige\ stop = i\right)}$$

$$\text{en } \Pr(huidige\ stop = i) = \sum_k \Pr\left(volgende\ stop = k, huidige\ stop = i\right)$$

waarbij k over alle mogelijke stops itereert. Empirisch wordt dit berekend aan de hand van tel f_{ij} voor de specifieke transitie tussen i en j. De kans wordt dan:

$$\Pr(volgende\ stop = j \mid huidige\ stop = i) = \frac{f_{ij}}{\sum_k f_{ij}}$$

Aangezien er mogelijks niet genoeg data zou zijn om de transitie kansen te bereken aan de hand van te tellen wordt de Laplace smoothing toegepast. De formule wordt dan als volgt:

$$\Pr(volgende\ stop = j \mid huidige\ stop = i) = \frac{f_{ij} + \alpha}{\sum_k f_{ij} + \alpha d}$$

waarbij α de smoothing parameter is (bij $\alpha = 0$ word er geen smoothing toegepast) en d het totaal aantal stops is.

Verder worden deze kansen gecombineerd met afstandskansen. De afstandskansen zijn invers proportioneel met de afstanden zelf. Ze kunnen ook bijgehouden worden in een matrix $D = [d_{ij}]$ net als bij transitiekansen. Hierbij is d_{ij} de afstandskans voor de overgang van stop i naar stop j. Deze kans wordt als volgt berekend:

$$d_{ij} = Pr_{dist}(volgende\ stop = j, huidige\ stop = i) = \frac{c'_{ij}}{\sum_k c'_{ik}}$$

$$\text{met } c'_{ij} = \frac{\sum_k c_{ik}}{c_{ij}}$$

Hier is c_{ij} de standard kost om van stop i naar stop j te gaan. c'_{ij} is de inverse van de relatieve kost. Het kan ook als volgt herschreven worden:

$$d_{ij} = \frac{c_{ij}^{-1}}{\sum_k c_{ij}^{-1}}$$

De combinatie van transitiekansen en afstand kansen wordt dan gedaan door een lineaire combinatie van de negatieve logaritmen van beide matrices T en D te nemen. De uitkomst is dan $T' = [t'_{ij}]$ waarbij:

$$t'_{ij} = -(\beta \log(t_{ij}) + (1 - \beta) \log(d_{ij}))$$

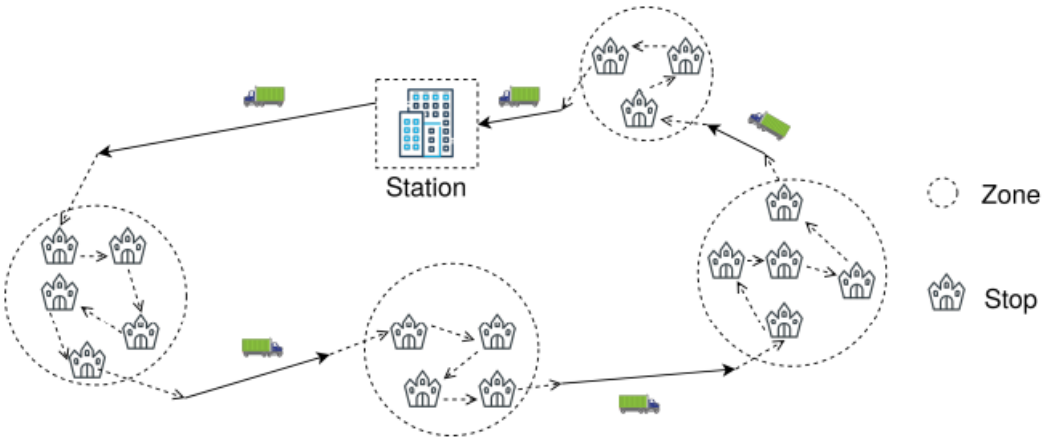
β is hier een parameter om de lineaire combinatie te controleren. Als $\beta = 1$ dan worden alleen de transitiekansen gebruikt, terwijl als $\beta = 0$ dan worden alleen de afstandskansen gebruikt. De matrix T' kan dan als input worden gegeven aan een TSP solver om een route te laten genereren.

Een groot voordeel van deze aanpak is dat het heel algemeen is. Er hoeft geen analyse van het dataset gedaan te worden. Er moeten geen patronen, beperkingen en bijhorende penalty's gezocht worden. De objectief functie die geminimaliseerd moet worden hoeft ook niet geconstrueerd te worden. Dit kan veel tijd en moeite besparen. Dit gebeurt omdat de patronen en beperkingen impliciet aanwezig zullen zijn bij de transitiekansen omdat ze gewoon vaker zullen voorkomen bij het tellen. Nadelen van deze aanpak zijn dat er natuurlijk veel data ter beschikking moet zijn om de onderliggende kansverdelingen goed te kunnen benaderen en dat er alleen maar eerste-orde patronen in de kansverdelingen aanwezig kunnen zijn. Dit komt omdat het tellen gebeurd voor elke individuele transitie en het onderliggend Markov model is een eerste orde model. Patronen en beperkingen die opereren op meer dan een individuele transitie kunnen daarom niet gedetecteerd worden door de kansverdelingen.

2.7.2 Markov model voor de routing challenge

Deze sectie wordt besproken aan de hand van Canoy et al. (2022).

Er wordt gebruik gemaakt van dit framewerk in Canoy et al. (2022) als aanpak voor de Last Mile routing challenge. Een twee staps algoritme wordt beschreven waarin de verantwoordelijkheden van de twee stappen worden opgesplitst. In de eerste stap is het doel om een routing te vinden op het zone niveau. In de tweede stap wordt de gevonden routing van de eerste stap gebruikt om een routing te vinden op het niveau van de stops. Het probleem in zijn geheel wordt dus in twee consequente stappen aangepakt. Het idee achter de 2-staps methode wordt getoond op Figuur 9 uit Canoy et al. (2022).



Figuur 9 : Visualisatie van 2-staps methode

De eerste stap maakt gebruik van het MLR framewerk. De waargenomen stops volgorden worden herformuleerd als zone volgorden waarbij opeenvolgende stops die bij dezelfde zone behoren samen worden gebundeld tot één zone. Zo wordt op het einde de waargenomen zone volgorde verkregen. Sommige zones zouden meerdere keren kunnen voorkomen in de zone volgorde. Alleen hun eerste bezoek wordt bijgehouden en de resterende bezoeken worden uit de volgorde verwijderd. Op deze manier wordt een zone volgorde verkregen voor elke route waarbij elke zone uniek is in dezelfde volgorde.

Deze volgorden worden vervolgens gebruikt om de probabilistische transitiematrix op te stellen. De telmatrix is dan een $n \times n$ matrix waarbij n het aantal zones is die over alle routes bezocht worden. Wanneer een nieuwe route voorspeld moet worden, dan worden alle zones uit die route gehaald en er wordt een transitiematrix opgesteld tussen al die zones. Hiervoor worden de overeenkomende elementen uit de telmatrix gebruikt. Er wordt dan een Laplace smoothing van 1 toegepast op alle tellen bij de normalisatie van de rijen om te voorkomen dat er gedeeld wordt door 0 of dat er kansen van 0% zouden zijn. Kansen met 0% kunnen namelijk verder voor problemen zorgen bij het nemen van de negatieve logaritmen van de elementen van de transitiematrix.

Er worden ook afstanden geïntegreerd in het model zodat er een lineaire combinatie kan worden genomen van de transitiekansen en de afstandskansen. Maar afstanden tussen zones worden niet gegeven. Hiervoor wordt dus een benadering gebruikt aan de hand van de exacte geografische coördinaten van de stops binnen elke zone (met lengte-en breedtegraden). Voor elke nieuwe route worden de coördinaten van alle stops die zich in dezelfde zone bevinden verzameld. Er wordt dan voor elke zone de gemiddelde locatie van de stops berekend en die locatie wordt dan het zwaartepunt van de zone. Afstanden tussen zones worden vervolgens berekend door de Euclidische afstand te nemen tussen de zwaartepunten. Zo kan een afstandenmatrix opgesteld worden voor de zones. Dit is ook een dynamische aanpak want het zwaartepunt van een zone kan veranderen afhankelijk van de stops die in de huidige route aanwezig zijn, ze worden dus niet op voorhand bepaald en vastgelegd. De afstandenmatrix wordt vervolgens omgevormd tot

een probabilistische matrix door eerst een Laplace smoothing van 1 toe te passen, dan door alle afstanden te inverteren en tot slot de rijen te normaliseren.

Een lineaire combinatie van de twee matrices kan dan als input aan een TSP solver worden gegeven zodat het de MLR kan voorzien. Om de beste β te vinden voor de lineaire combinatie van transitie-en afstandskansen te zoeken wordt gebruik gemaakt van een Structured Output Prediction (SOP) algoritme. De representatie functie voor SOP is $\varphi(u,x)$ waarbij u de input van het probleem voorstelt en x de ideale output. Er worden twee gewichten gezocht in een vector $w = [w_d, w_t]$ zodat $\max w^T\ \varphi(u,x)$ overeenkomt met :

$$\min \sum_{ij} -w_d \log(d_{ij}) - w_t \log(t_{ij})$$

Hieruit volgt dan dat:

$$\varphi(u,x) = \left[\sum_{ij} \log(d_{ij}) x_{ij}\,, \sum_{ij} \log(t_{ij}) x_{ij}\right]$$

Aan de hand hiervan wordt de w vector getraind door ze te updaten na elke iteratie met de volgende regel:

$$w \leftarrow w + \delta(\varphi(u,x) - \varphi(u,\hat{x}))$$

waarbij δ de learning rate is en $\hat{x}$ de voorgestelde oplossing door TSP solver is. Aan de hand van dit proces worden w_d en w_t berekend en β kan dan gezien worden als de verhouding tussen w_t en w_d. Dit proces werd 5 keer over de hele training set uitgevoerd. Na de eerste epoch werden de optimale gewichten gevonden $w_d = 1.58$ en $w_t = 2.33$ die in een zone score van 0.0289 resulteerden. De initiële waarde voor beide gewichten was 1. Als allen de afstandskansen gebruikt werden hiervoor dan was de zone score 0.0642 geweest terwijl als alleen de transitiekansen gebruikt zouden zijn geweest dan werd er een zone score van 0.0374 behaald. Indien ze alle twee zouden gebruikt zijn maar de gewichten niet geoptimaliseerd waren met SOP en beide op 1 waren gebleven dan was de zone score 0.0291 geweest.

Hier wordt de zone score berekend en wordt als maat gebruikt voor de eerste stap. Deze wordt berekend net als de stop score, gebruik makend van dezelfde functie , maar in plaats van de reistijden te gebruiken als maat worden de berekende afstanden tussen de zones gebruikt.

In de tweede stap wordt een TSP solver gebruikt om de stopscore te vinden. Om ervoor te zorgen dat de gevonden zone volgorde gerespecteerd wordt, bij het zoeken van de stop volgorde, worden er beperkingen geconstrueerd op basis van de zone volgorde. Er zijn in totaal 7 verschillende beperkingen. De eerste stelt een penalty voor wanneer twee stops zich in dezelfde zone bevinden. De tweede en de vierde zijn voor wanneer twee stops zich in buurzones bevinden, dus zones die vlak voor of achter elkaar komen in de zone volgorde. De derde en de vijfde zijn voor als de stops zich in zones bevinden dat twee plaatsen van elkaar verwijderd zijn en de zesde voor als ze drie of meer plaatsen

ver van elkaar zijn. Dan is er nog een ‘nulde’, deze stelt het gewicht voor van de reistijden. De totale kost wordt dan:

$$c_{ij} = w_0 tt_{ij} + \sum_{r=1}^{6} w_r f_r(i,j)$$

Hierbij stellen tt_{ij} de reistijden voor en $f_r(i,j)$ de booleaanse functie die bij de r-de beperking hoort en die als output 1 of 0 kan geven. De beperkingen sluiten elkaar af, dit wilt zeggen dat er per stop paar i,j telkens maar een beperking waar kan zijn.

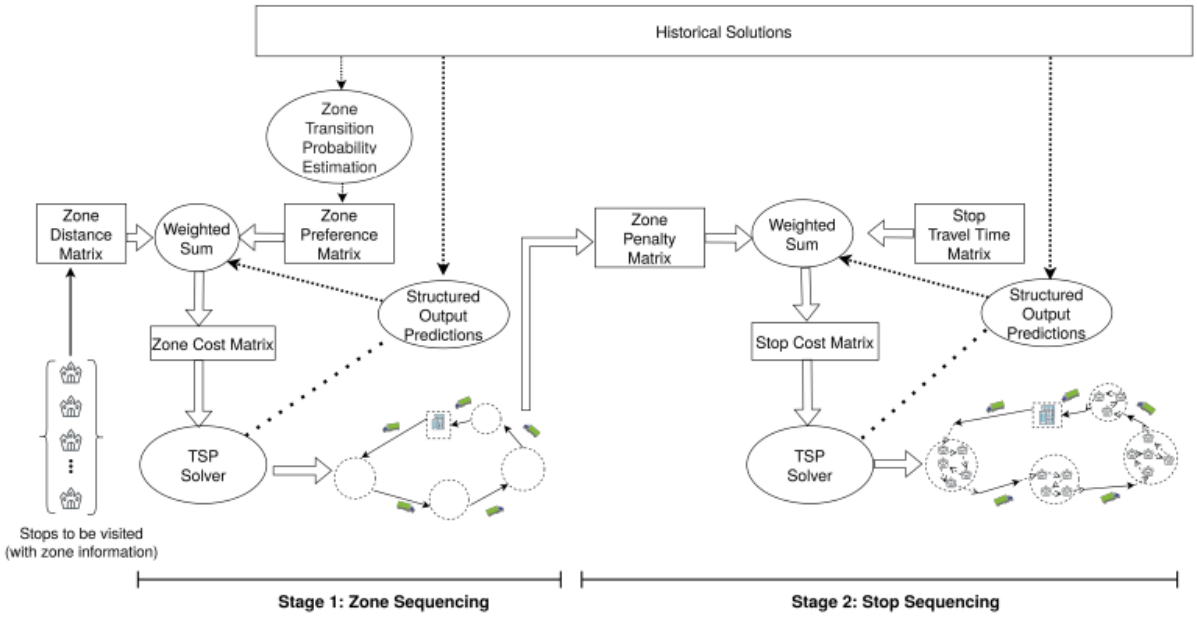
De gewichten werden opnieuw aan de hand van SOP gezocht. De gevonden waarden zijn:

$$[w_0, w_1, w_2, w_3, w_4, w_5, w_6, w_7] = [2, 0.35, 2.13, 4.09, 2.32, 4.02, 6.10]$$

En resulteerden in een stop score van 0.0465. Als er alleen afstandskansen zouden gebruikt zijn dan was de stop score 0.0625 geweest en als er alleen transitiekansen gebruikt werden, dan was de stop score 0.0479 geweest.

De gebruikte TSP solver was de Google-OR-Tools TSP Solver (*Traveling Salesperson Problem*, z.d.). Voor de eerste stap werd deze gebruikt met een tijdslimiet van 1 second en gebruikt makend van de First Solution Search heuristiek. In de tweede stap werd er gebruik gemaakt van een Guided Local Search heuristiek die voor 30 seconden per instantie mocht zoeken. De train/validatie split werd gemaakt door de laatste 544 hoge kwaliteit routes van de dataset als validatie set te gebruiken.

Op Figuur 10 uit Canoy et al. (2022), wordt een compleet digamma getoond van de besproken 2-staps methode.



Figuur 10 : Diagram van 2-staps methode

2.8 Het gebruik van Neurale Netwerken

In deze sectie wordt het gebruik van neurale netten besproken en hoe bepaalde architecturen en technieken gebruikt zouden kunnen worden om de Last Mile Challenge aan te pakken. Dit word gedaan door algemene NN technieken te bespreken.

2.8.1 Generatieve modellen en seq2seq

In recente jaren zijn er veel ontwikkelingen gebeurd in verband met generatieve neurale netten (GNN). Deze vooruitgangen gebeurden voornamelijk in het veld van natural language processing (NLP). In dit veld worden veel seq2seq modellen ontworpen, gebruikt en bestudeerd. Het principepen van een seq2seq model is dat het een volgorde als input neemt en een volgorde als output kan genereren. De meeste modellenzijn gebaseerd op een encoder en decoder architectuur. Alle elementen in de input volgorde worden gecodeerd zodat het model zich een idee kan vormen over de globale context. Vervolgens wordt de decoder op recursieve manier gebruikt om outputs te genereren die samen de outputvolgorde vormen. Varianten van recurrent neural networks (RNN) worden hier vaak voor gebruikt om volgorden te kunnen coderen en genereren.

Een voorbeeld van types RNN's dat gebruikt worden zijn long-short term memory networks (LSTM's) van Sutskever et al. (2014). Het concept van LSTM's is dat elke LSTM-cel een cel staat heeft dat gebruikt kan worden als een soort "geheugen" over wat tot op een bepaald punt in de volgorde is gezien geweest. Dit is ontworpen zodat het probleem van verkleinende gradiënten aangepakt wordt. Zonder dit mechanisme worden gradiënten die berekend worden voor backpropagation bij het terugkeren van de tijd (of volgorde van de individuele inputs) kleiner en kleiner worden. Aangezien het geheugen

element (of de staat) van een LSTM-cel gebruikt wordt bij het bepalen van de huidige output, wordt het probleem van de gradiënten omzeild.

Het attention mechanisme in Vaswani et al. (2017) is ook een succesvolle variant van RNN's. Het ligt aan de basis van de Transformer-architectuur. Het mechanisme kan worden beschreven als een werking tussen een query (Q) en een key-value (K-V) paar waarbij Q,K en V vectoren zijn. De output van dit mechanisme is een gewogen som van de waarden V waarbij de gewichten vector berekend wordt aan de hand van een compatibiliteitsfunctie die werkt op Q en K. In een Transformer-architectuur is deze functie een dot-product.

In de meeste architecturen wordt een input volgorde gemapt op een output volgorde maar dit is niet noodzakelijk wat er in de Last Mile challenge moet gebeuren. Om TSP oplossingen te simuleren moet de output een herordening zijn van de input. Dit probleem wordt aangepakt door Vinyals et al. (2015) aan de hand van Pointer netwerken. Het idee achter dit soort netwerken is dat de output geselecteerd wordt van de inputs door een pointer die bij elke decodering aan elke input een waarschijnlijkheid toewijst. De input op de positie die de hoogste waarschijnlijkheid krijgt toegewezen wordt dan geselecteerd als output voor die decodering. Om de kansen te berekenen wordt hier ook gebruik gemaakt van het attention mechanisme. In hun werk tonen ze aan met Ptr-Net dat TSP's tot een grootte van 50 steden opgelost kunnen worden en bijna even goed kunnen zijn als traditionele solvers. Een probleem dat werd ondervonden was dat de volgorde waarin de inputs aan het model werden gegeven een invloed speelde bij de resultaten. Dit is natuurlijk een probleem aangezien een ideale TSP solver altijd dezelfde output zou moeten geven voor dezelfde set van inputs of voor alle mogelijke permutaties van de volgorde van de inputs. Dit probleem wordt aangepakt in Vinyals et al. (2016) aan de hand van een set pooling mechanisme. Een 'proces' block dat bestaat uit LSTM-cellen blijft geüpdatet worden door een 'reading' block die de inputs leest en ze embedt in geheugen vectoren. De geheugen vectoren worden dan naar het proces block gestuurd die recursief het attention mechanisme gebruikt op de geheugenvectoren. Op het einde is de staat een embedding van de input set die dus permutatie invariant is.

Over het algemeen hebben seq2seq Transformer modellen enorm veel data nodig. In 'Attention is All You Need' van Vaswani et al. (2017) word het model voor Engels-Duits met 4.5 miljoen vertaalde voorbeeld zinnen getraind en voor het Engels-Frans model wordt getraind met 36 miljoen zin paren. Om Ptr-Net, die TSP's kan oplossen, van Vinyals et al. (2015) te trainen worden 1 miljoen voorbeelden gebruikt. Dit is een probleem aangezien er voor de Last Mile routing challenge maar 6,112 training routes ter beschikking zijn.

2.8.1 Discriminerende modellen

Een alternatief voor generatieve seq2seq modellen zijn modellen die stap per stap werken. Dit is een model dat iteratief wordt opgeroepen op telkens de volgende stop te kiezen. Vervolgens wordt de gekozen stop terug als input gegeven om nog eens de volgende te kiezen. Zo kan het model een volledige volgorde genereren. Het probleem hiervan is dat het model een greedy heuristiek zou simuleren door telkens voor elke stap de meest waarschijnlijke transitie te kiezen en niet naar het geheel te kijken.

In Wang et al. (2020) wordt een architectuur hiervoor voorgesteld, namelijk een Alternative Specific Utility DNN (ASU-DNN). In tegenstelling tot een volledig verbonden DNN waarbij de utiliteit van een 'alternatief' (in dit geval een transitie) berekend wordt met alle andere alternatieven, wordt de utiliteit van een alternatief berekend op basis van het alternatief zelf. In theorie zou dit voor een lagere estimation error moeten zorgen aangezien de architectuur minder connecties (en dus parameters) heeft dan een normale DNN architectuur. Maar dit zou natuurlijk leiden tot een grotere approximation error. Een ander groot voordeel van de ASU-DNN architectuur is de interpreteerbaarheid. Aangezien elk alternatief apart wordt geëvalueerd en ze vervolgens met elkaar worden vergeleken is het gemakkelijk om te begrijpen waarom welke alternatieven gekozen zijn geweest ten opzichte van anderen, hun utiliteiten kunnen direct waargenomen worden.

Een manier om gebruik te maken van deze architectuur is om de utiliteit van elke mogelijke transitie apart te berekenen en ze vervolgens door te geven aan een softmax layer. Er moet niet meteen een keuze gemaakt worden. De kansverdeling kan worden bijgehouden en alle mogelijke kansverdelingen vanuit elke mogelijke source kunnen berekend worden. Uiteindelijk kan de volgorde van transities worden gekozen aan de hand van een transitie matrix met een kansendistributie op elke rij. Op deze manier wordt er geen greedy heuristiek gesimuleerd.

In Mandi et al. (2021) wordt er in een MLR framewerk gebruik gemaakt van een gelijkaardige architectuur. In dit werk worden de transitie kansen geleerd door een ASU-NN in plaats van door Markov counting. De dataset bevat 201 dagelijkse routes over een span van 39 weken en 73 unieke stops voor een VRP. De inputs zijn embeddings van de stops dat geserveerd moeten worden, logaritmen van de afstandskansen, een embedding van de huidige dag van de week, een lineaire lookback rekening te kunnen houden met oude oplossingen en de logaritmen van de Markoviaanse transitiekansen. De gebruikte loss functie is een categorical cross-entropy (CE) loss, het probleem wordt dus aangepakt als een classificatie. De architecturen die zijn getest zijn een DNN, LSTM en linear layers. In de architectuur met linear layers wordt een verschillende linear layer gebruikt voor elke 'source' node, dus voor elke stop waaruit de transitie vertrekt. In de testen bleek dit voor overfitting te zorgen met een heel lage training loss maar een hoge test loss.

Omdat een betere CE loss zich niet noodzakelijk vertaalt naar betere TSP oplossingen wordt er ook getest met decision focussed learning. Hierbij wordt een regret functie gedefinieerd en het netwerk leert uit die regret functie. De regret functie is dan gebaseerd op de resultaten van de TSP solver. Het model kan zo dus direct uit de outputs van de TSP solver leren en de structuur van het onderliggend probleem beter integreren bij het leerproces. In de resultaten van de experimenten bleek het geen goede oplossingen te genereren, wat mogelijks veroorzaakt kan worden door het feit dat er weinig data beschikbaar was.

2.9 Conclusie literatuurstudie

Er is een groot aanbod aan TSP solvers waaruit gekozen kan worden. TSP solvers zouden dus geen probleem moeten zijn voor de Last Mile Routing Challenge. Sommige teams hebben hun eigen solver opgebouwd. Dit heeft als voordeel dat ze eenvoudig custom penalty functies konden definiëren en bijvoegen aan de objectief functie. De meeste teams die dit doen starten vanuit bestaande framewerken zoals een al bestaand genetisch algoritme of een al bestaand LKH solver.

Belangrijke elementen uit de dataset analyses zijn dat de zone ID's en hun symbolen essentieel blijken te zijn. De symbolen veranderen met de volgende prioriteiten: eerst het 3de symbool, dan het 4de dan het 2de en tot slot het 1ste. De teams brengen dan hun variaties van de zone ID beperkingen in hun objectief functies waarbij de penalty's voor het veranderen van een symbool toenemen naarmate het symbool een lagere prioriteit heeft. Aan de basis van de objectief functie liggen natuurlijk de reistijden.

Het kiezen van de richting waarin een volgorde uitgevoerd moet worden blijkt een terug komend probleem te zijn. In Guo et al. (2023) wordt het probleem aangepakt in een postprocessing fase door eerst stops te laten serveren die grote pakketten hebben en waarbij de levertijd lang duurt. Cook et al. (2021) lossen het probleem op aan de hand van voorrang beperkingen gebaseerd op een referentieroute.

Een suggestie van Guo et al. (2023) is om het probleem aan te pakken door de eerste en de laatste zones zo goed mogelijk te voorspellen aangezien dat de meerderheid van de routes heel consequent het pad dat de zone ID beperkingen respecteert volgen. Maar uit de dataset analyses blijkt dat de eerste en de laatste zones heel moeilijk zijn te voorspellen aan de hand van afstanden. Vaak moeten de bezorgers namelijk snelwegen gebruiken om van het station naar de stops te geraken. Welke dat er eerst geserveerd worden hangt dan volgens Guo et al. (2023) af van de ligging van op- en afritten.

MLR met Markov counting blijkt een goede aanpak te zijn in Canoy en Guns (2019). Het voordeel ervan is dat er geen dataset analyse moet worden gedaan omdat de onderliggende patronen impliciet door het eerste orde transitie model zullen gemodelleerd worden. Deze ideeën worden dan verder gezet in Canoy et al. (2022) in de context van de Last Mile Routing Challenge. Het transitie model wordt opgebouwd op het niveau van de zone ID's en dus in de eerste stap van het twee staps algoritme. Aangezien theoretische resultaten uit Cook et al. (2021) aantonen dat het probleem praktisch opgelost is als de volgorde van de zone ID's gekend is, komt dit twee staps algoritme goed uit. Het heeft dus zin om van een zone volgorde te spreken. Op het einde van Canoy et al. (2022) stellen de auteurs voor om het transitie model op te stellen aan de hand van een neuraal netwerk in plaats van Markov counting.

Het gebruik van een zone score heeft als voordeel dat er een maat is om de eerste stap te kunnen evalueren. Het nadeel is dat het niet noodzakelijk een accurate maat is. Eerst en vooral omdat kleine fouten in de eerste stap voor grote fouten zorgen in de tweede stap. Het is duidelijk uit de resultaten dat de zone scores niet overeenkomen met de stop scores. Ten tweede omdat zone afstanden worden gebruikt om de zone score te

berekenen wat een zelf gedefinieerd afstandsmaat is. Het is namelijk ook een symmetrische maat wat inhoudt dat de afstand tussen twee zones dezelfde is in beide richtingen. Dit zou mogelijks consequenties kunnen hebben voor het bepalen van richtingen.

Het gebruik van generatieve seq2seq modellen is theoretisch heel aantrekkelijk maar lijkt praktisch onhaalbaar met de beperkte hoeveelheid data ter beschikking. Discriminerende modellen lijken meer gepast voor de hoeveelheid data. ASU-DNN's kunnen gebruikt worden om een utiliteit toe te kennen aan elke mogelijke keuze. Vervolgens kunnen de berekende utiliteiten gebruikt worden om ze tot distributies om te vormen. Dit mechanisme kan toepasselijk zijn voor keuzes te maken tussen de mogelijke transities.

In Mandi et al. (2021) wordt dit concept succesvol toegepast op een VRP. Het probleem wordt aangepakt als een classificatie probleem en er wordt gebruik gemaakt van CE loss om de netwerken trainen.

De target distributies zijn discrete distributies die een kans van 100% toekennen aan de transitie die werkelijk gebeurt. Er wordt een assumptie gemaakt dat een verbetering in CE loss ook een verbetering in TSP oplossingen voorstelt. Om die assumptie niet te moeten maken wordt er geprobeerd om het netwerk aan de hand van decision focussed learning te trainen maar dit leverde slechtere resultaten.

Een ander idee dat werd getest was om verschillende linear layers te gebruiken voor elke verschillende bron van een transitie. Deze methode bleek helaas voor overfitting te zorgen.

Hoofdstuk 3: Methodologie

3.2 Theoretische experimenten

Er wordt verder gewerkt op de 2-staps aanpak van Canoy et al. (2022) en de principes van Canoy en Guns (2019). Dit omdat het een aanpak is waarbij het niet nodig is om verdere dataset analyses uit te voeren aangezien het Markoviaans transitie model de eerste orde patronen impliciet aanleert. Zoals al waargenomen werd door de autheurs in Cook et al. (2021) zou een stop score van 0.0077 mogelijk zijn als de zone volgorde perfect gekend is. De 2-staps aanpak steunt daarop. Als de eerste stap perfect zou verlopen dan zou er in de tweede stap een heel lage stop score bereikt kunnen worden. Dit experiment wordt eerst getest op het 2-staps algoritme, er wordt namelijk gekeken of dat het ook in graad is om een gelijkaardige score te halen indien de eerste stap ideale zone volgordes zou leveren.

Enkele andere experimenten die gebaseerd zijn op traditionele aanpakken worden ook uitgevoerd. Eerst worden de stop volgordes gegenereerd aan de hand van de TSP solver door gewoon de reistijden te minimaliseren. Vervolgens wordt er een clustered ATSP aanpak genomen door een grote constante bij de reistijden op te tellen indien ze zich niet in dezelfde zone zouden bevinden. Dan worden er theoretische experimenten uitgevoerd op specifiek de 2de stap. Hiervoor worden de perfecte zone volgorden gebruikt. Eerst wordt deze zone volgorde gerespecteerd aan de hand van een clustered ATSP algoritme zodat er penalty's worden toegepast op stops van verschillende zones maar niet indien de zones buurzones zijn in de zone volgorde:

$$f(i,j) = \begin{cases} 0, & index(j) - index(i) = 1 \ \vee index(i) = index(j) \\ 1, & anders \end{cases}$$

Met deze booleaanse functie wordt er geen penalty toegepast wanner twee stops i en j uit zones komen die elkaar opvolgen. Met deze formulering wordt de richting van de volgorde gerespecteerd. Indien er absolute tekens geplaatst zouden worden als volgt:

$$|index(j) - index(i)| = 1$$

Dan wordt de richting bepaald door de TSP solver die de snelste richting zal kiezen. Verder wordt er ook geprobeerd met een penalty vermenigvuldiger van 0.5 bij het volgen van de richting (in plaats van 0, geen penalty). Tot slot wordt het geprobeerd op een versimpelde versie van de gevonden gewichten van het 2-staps algoritme. Aangezien ze ongeveer symmetrisch zijn worden ze als volgt versimpeld:

$$f(i,j) = \begin{cases} 0, & index(i) = index(j) \\ 1, & index(j) - index(i) = 1 \\ 2, & index(j) - index(i) = 2 \\ 3, & ianders \end{cases}$$

Ten opzichte van het origineel 2-staps algoritme zijn de gewichten voor dezelfde violaties in beide richtingen samengenomen en vervangen door de dichtstbijzijnde integer

waarde. Vervolgens zijn ze gedeeld door 2 aangezien het gewicht dat voor de reistijden een 2 was. Op deze manier wordt het gewicht voor de reistijden een 1. Opnieuw, door absolute waarden te plaatsen rond het verschil van de indices bij de casussen wordt er geen richting geforceerd en kiest de TSP solver zelf.

3.3 Andere manieren van tellen

In het origineel 2-staps algoritme worden transities geteld om een telmatrix op te bouwen. In praktijk bestaat deze telmatrix uit meerdere submatrices voor elke station ID. Wanneer de transities van een route geteld worden, worden ze geattribueerd aan de bijhorende station ID matrix zodat het overeenkomt met het station waaruit de bezorger is vertrokken. In totaal zijn er 17 verschillende station ID's. Figuur 11 toont hoeveel verschillende zone ID's er aanwezig zijn in de training set per station ID. Aan de hand hiervan kan ook gezien worden hoe groot elke submatrix is en ook dat er een totaal aantal unieke zones is van bijna 36,000. Dit wilt niet zeggen dat ze allemaal verschillende zone ID's hebben. Er kunnen namelijk meerdere zones zijn met dezelfde ID die zich in verschillende stedelijke ruimten bevinden. Er zijn in totaal 5 verschillende stedelijke ruimten: Los Angeles, Chicago, Seattle, Boston en Austin. De eerste 3 letters van een station ID komen overeen met de stedelijke ruimte waarin de station zich bevindt.

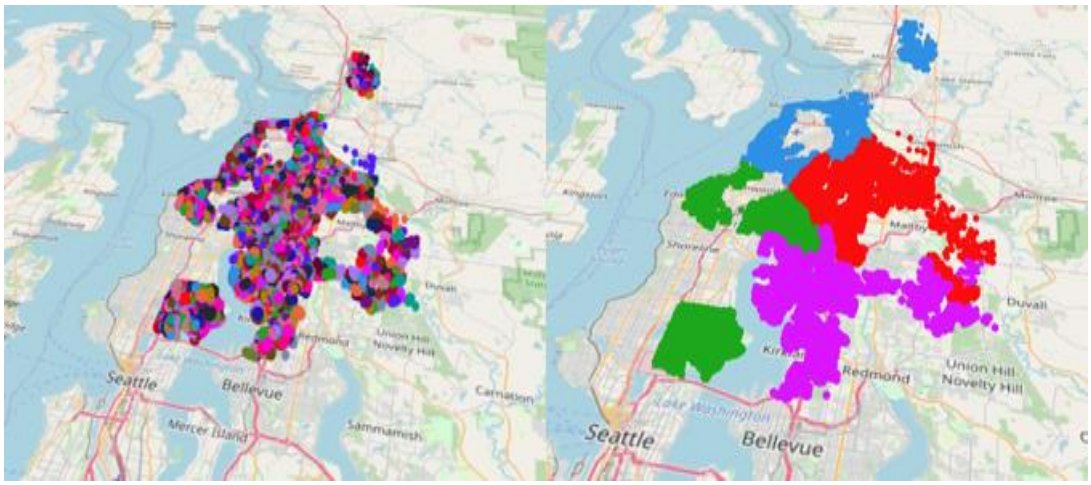
City	Stations						Totals
Los Angeles	DLA3	DLA4	DLA5	DLA7	DLA8	DLA9	
	1388	1092	1353	4294	3550	4736	16413
Chicago	DCH1	DCH2	DCH3	DCH4			
	1754	1278	1862	2286			7180
Seattle	DSE2	DSE4	DSE5				
	541	1783	1236				3560
Boston	DBO1	DBO2	DBO3				
	549	2274	3309				6132
Austin	DAU1						
	2653						2653
							35938

Figuur 11 : Aantal zone ID's per station ID en station ID's per stedelijke ruimte

Voor deze reden wordt een nieuwe ID gedefinieerd. Deze ID zal unieke ID genoemd worden omdat het uniek is voor elke verschillende zone ten opzichte van de stations. Het unieke ID wordt gedefinieerd als de concatenatie van de station ID en de zone ID. In totaal zullen er dan ongeveer 36,000 verschillende unieke ID's zijn in de training set. Terwijl er maar 8990 unieke zone ID's in de training set aanwezig zijn.

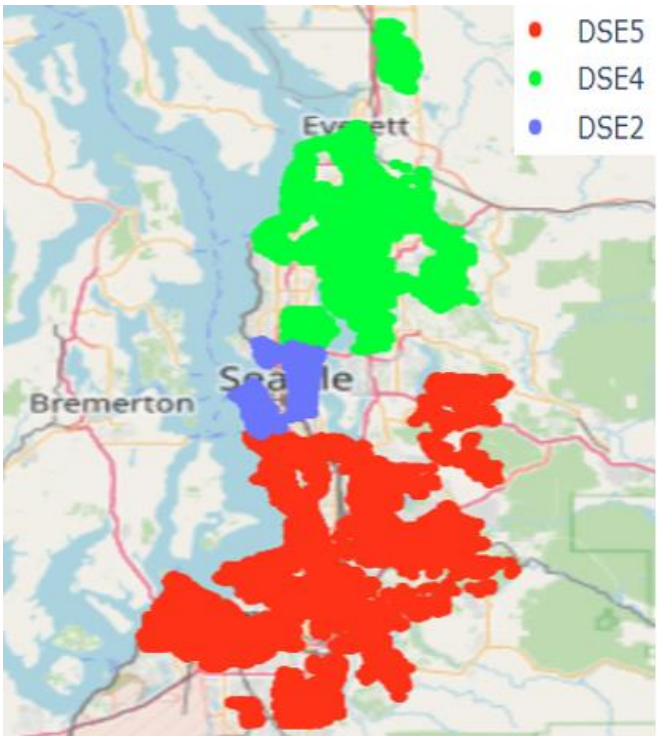
Op

Figuur 12 kunnen alle stops gezien worden die bereikt worden door station DSE4 in het stedelijk gebied van Seattle. Stops zijn ingekleurd op basis van hun zone ID's, dezelfde kleuren moesten voor meerdere zone ID's gebruikt worden.



Figuur 12 : Alle stops bereikt door DSE4 ingekleurd per zone ID (links) en de zone ID's geclusterd op basis van hun eerste symbool (rechts)

Er kunnen ook geografische ID's gedefinieerd worden. Deze zijn gedefinieerd door een combinatie te nemen van het stedelijk gebied (of de eerste 3 letters van een station ID) en de zone ID. Geografische ID's komen perfect overeen met de geografische zones in de werkelijkheid. Dit is bij unieke ID's niet het geval omdat sommige geografische zones geserveerd worden door meerdere stations. Op Figuur 13 worden alle stops die bereikt worden in het stedelijk gebied van Seattle getoond. Verschillende stations kunnen dezelfde zone ID serveren binnen hetzelfde stedelijk gebied, dit gebeurt vooral aan de randen van hun bereik.



Figuur 13 : Alle stops in het stedelijk gebied van Seattle. Verschillende kleuren worden gebruikt om het bereik van verschillende stations te illustreren

Het 2-staps algoritme van Canoy et al. (2022) construeert eerst de submatrices op basis van wat overeenkomt met hun unieke ID's. Bij het construeren van een telmatrix voor een nieuwe set van zones worden de rijen van die zones voor elke submatrix opgeteld waardoor het dan praktisch gebruik maakt van de zone ID's. Dit is equivalent met een 8990 x 8990 matrix te construeren gebaseerd op transities tussen zone ID's en de relevante elementen van deze matrix dan te gaan gebruiken wanneer er een voorspelling gemaakt moet worden.

Het kan interessant zijn om voor een voorspelling van een zone volgorde van zones die zich in eenzelfde stedelijk gebied bevinden alleen te werken met de data van dat stedelijk gebied. Dit wordt ook getest door alleen de submatrix te gebruiken van dezelfde station als die van de route die voorspeld moet worden.

Een andere telmethode wordt ook getest. In plaats van de transities tussen twee zone ID's te tellen worden connecties tussen twee zone ID's geteld. Dit houdt in dat er geteld gaat worden hoe vaak twee zone ID's X en Y naast elkaar voorkomen in een waargenomen zone volgorde onafhankelijk van de volgorde waarin ze voorkomen. Om de transitie kans van X naar Y, $Pr(X \rightarrow Y)$ te benaderen wordt er geteld hoe vaak er in de waargenomen zone volgorden de deel deelrij $X \rightarrow Y$ aanwezig is. Om de connectie kans $Pr(X \leftrightarrow Y)$ te bepalen wordt er geteld hoe vaak de connectie $X \leftrightarrow Y$ aanwezig is in de waargenomen zone volgorden. Het idee erachter is dat er op deze manier een betere distributie kan benaderd worden omdat er 'meer data' zou worden gebruikt, het zou gezien kunnen worden als een vorm van data augmentatie.

In praktijk wordt dit gedaan aan de hand van de volgende relatie:

$$count(X \leftrightarrow Y) = count(X \rightarrow Y) + count(X \leftarrow Y)$$

waarbij $count(.)$ de telfunctie voorstelt die telt hoe vaak de input waargenomen wordt in de trainingstest. Op deze manier hoeven er weinig aanpassingen te gebeuren want alle nodige informatie is al aanwezig in de submatrices met de tellen van elke mogelijke transitie. Er moeten gewoon telkens twee elementen uit de nodige submatrix opgesomd worden om het tel te krijgen voor de connecties. Dit principe kan toegepast worden op submatrices op niveau van de unieke ID's maar ook op matrices gebaseerd op de zone ID's. De 4 verschillende mogelijkheden worden geprobeerd.

Een nadeel van het gebruiken van connectiekansen kansen $P(X \leftrightarrow Y)$ in plaats van $P(X \rightarrow Y)$ is dat er informatie over de richting waarin de route uitgevoerd moet worden verloren gaat. Connectiekansen zijn namelijk symmetrisch:

$$Pr(X \leftrightarrow Y) = Pr(Y \leftrightarrow X)$$

Dit heeft als gevolg dat er aan de TSP solver een symmetrische matrix wordt gegeven. Ook als ze opgeteld worden met de afstandskansen $Pr_{dist}(X \leftrightarrow Y)$ verandert er niets, want afstandskansen zijn ook symmetrisch. Doordat de gerichte graaf van de zone ID's als ongericht wordt gezien, is alle informatie over richtingen verloren gegaan.

Dit wordt niet opgemerkt door de zone score. Doordat de zone score aan de hand van symmetrische afstanden tussen de zones wordt berekend heeft dit als gevolg dat de zone score van een volgorde en haar geïnverteerde volgorde altijd gelijk zijn aan elkaar. De zone score van bijvoorbeeld de volgende volgorden $A \rightarrow B \rightarrow C$ en haar inversie $C \rightarrow B \rightarrow A$ waarbij A,B en C zone ID's voorstellen, is dezelfde.

De tellen gebeuren door hoge kwaliteit routes een gewicht van 3 te geven, gemiddelde routes een gewicht van 2 en lage kwaliteit routes krijgen een gewicht van 1. Op deze manier worden tellen van routes van betere kwaliteiten als belangijker beschouwd.

3.4 Referentie mechanisme

Om het probleem van richtingen aan te pakken wordt een referentie systeem gebruikt dat geïnspireerd is door die van Cook et al. (2021) en het postprocessing van Guo et al. (2023).

In Guo et al. (2023) wordt een gelijkaardig probleem ondervonden. Namelijk dat de voorgestelde volgorden vaak in de tegengestelde richting gaan. Dit heeft een impact op de uiteindelijke stop score. Er wordt voor gekozen om de richtingen van een voorgestelde volgorde aan te passen in een postprocessing fase aan de hand van enkele gevonden patronen. Deze patronen zijn dat pakketten die langere bezorgtijden hebben en grotere pakketten vaak eerst worden geleverd.

In Cook et al. (2021) wordt dit probleem niet vermeld. Dat komt waarschijnlijk door hun gebruik van voorrang beperkingen. Door een referentie te zoeken voor de huidige set

van zones en voorrangsbeperkingen te genereren aan de hand van die referentie wordt de richting van de route al bepaald.

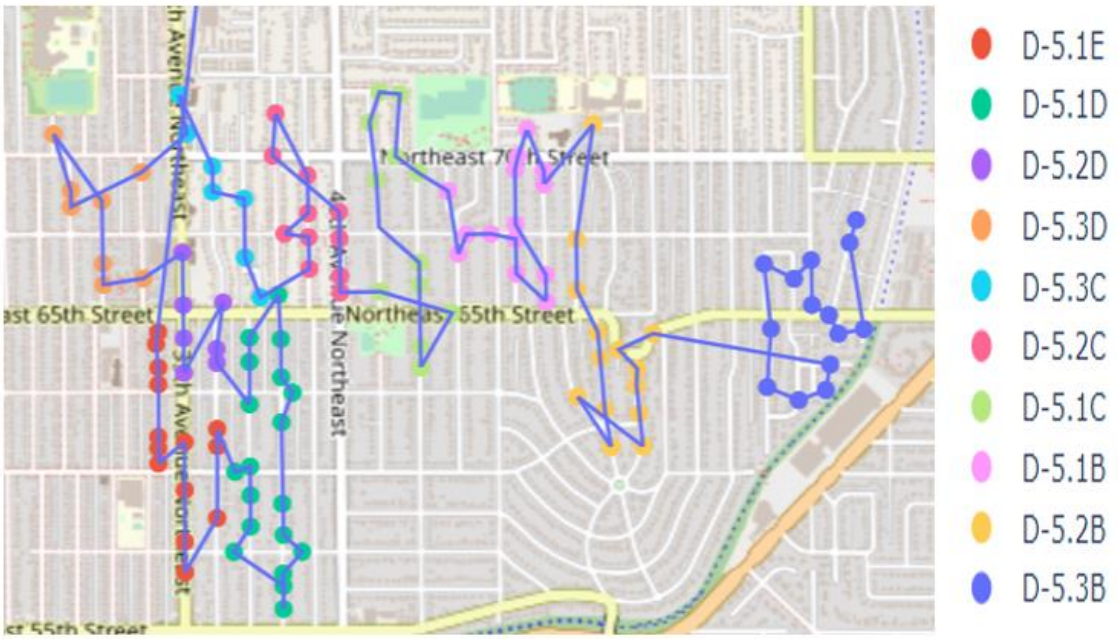
Er wordt een mix van beide aanpakken gebruikt. Nadat er een zone volgorde voorgesteld wordt door de TSP solver start er een postprocessing fase. In deze postprocessing fase wordt een referentieroute gezocht uit het training set voor de huidige route. Deze referentieroute wordt gezocht op bases van hoeveel gemeenschappelijke zone ID ze hebben. Bovendien wordt de referentie route gekozen uit de set van training routes die behoren tot hetzelfde stedelijk gebied.

Op Figuur 14 en Figuur 15 worden een voorbeeld route en haar toegewezen referentie route respectievelijk geïllustreerd. Alle zones van de voorbeeld route zijn te vinden in de referentieroute en moeten ook daar worden bezocht. Dit hoeft niet altijd het geval te zijn.

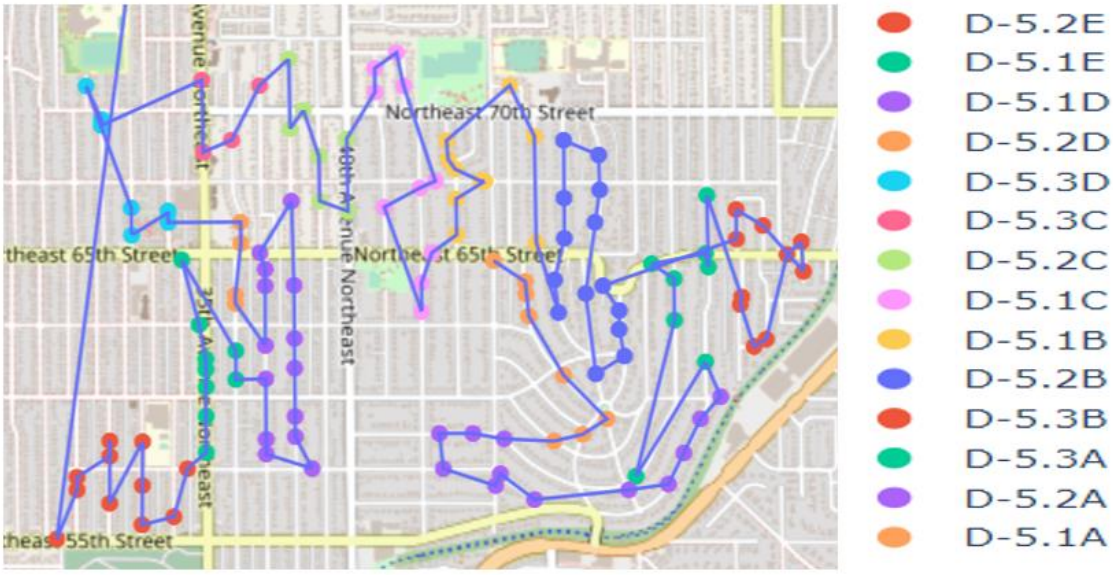
Uit de referentie route worden voorrang beperkingen gegenereerd van de vorm $index(X) < index(Y)$ waarbij X en Y verschillende zones zijn uit de set van gemeenschappelijke zones. Een counter wordt aangemaakt en die start op 0. Vervolgens wordt elke beperking gecheckt op de voorgestelde voorbeeld volgorde. Als de beperking gerespecteerd wordt, dan wordt +1 bij de counter opgeteld. Indien de beperking niet gerespecteerd wordt dan wordt er -1 op de counter gedaan. Als de beperkingen allemaal gecheckt zijn wordt er naar de staat van de counter gekeken. Als de counter positief eindigt dan mag de volgorde blijven zoals ze is voorgesteld. Indien de counter negatief zou zijn dan wordt de volgorde volledig geïnverteerd en wordt dat de nieuwe voorgestelde volgorde. De redenering hierachter is dat de richting wordt gekozen dat de meerderheid van de gegenereerde beperkingen respecteert. Als de counter negatief eindigt dan wordt de meerderheid gevioleerd en zou de geïnverteerde volgorde de meerderheid wel respecteren.

Indien de zone volgorden perfect worden voorgesteld kan dit mechanisme alleen tot extra fouten leiden. Dit zal dus ook getest worden op perfecte zone volgorden om te zien hoeveel fouten het maakt.

Hier wordt de assumptie gemaakt dat een perfecte zone volgorde geconstrueerd kan worden op basis van de connecties zonder enige informatie over de richtingen. De assumptie is dus dat de twee aspecten (connectie volgorde en richting) onafhankelijk van elkaar zijn.



Figuur 14 : Voorbeeld route om te evalueren aan de hand van referentie (start: links beneden in D-5.1E)



Figuur 15 : Referentie route voor voorbeeld route (start: links beneden in D-5.2E)

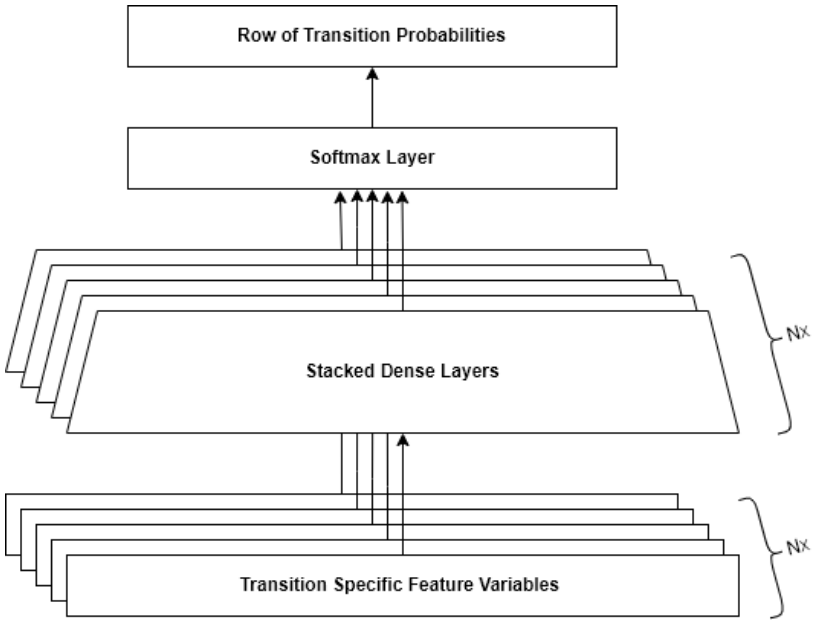
3.5 NN voor distributies te veralgemenen

De aanpak aan de hand van MLR werkt goed als er veel data beschikbaar is. Veel data ter beschikking hebben is noodzakelijk om een robuust eerste orde Markov model op te stellen. Dit is helaas niet het geval bij de Last Mile Routing Challenge omdat er maar 6,112 routes ter beschikking zijn en er maar 5,568 in de training set zijn die gebruikt kunnen worden om het model op te stellen. Een ander probleem is dat het aantal zones dat bezocht moeten worden niet vast ligt. Er kunnen zones geëvalueerd moeten worden

die nog nooit eerder gezien zijn geweest op vlak van hun zone ID, dus een zone met een zone ID die nog nooit eerder is gezien, of unieke ID, dus een zone die binnen een bepaald stedelijk gebied nog nooit eerder is gezien. Afhankelijk van welke manier van tellen er wordt toegepast en welke kansen distributie wordt gebruikt zullen er verschillende 'gaten' in de distributies zijn. Gaten kunnen beschouwd worden als transities waarover geen data verzameld is en die dus een uniforme kans toegewezen krijgen door de toegepaste smoothing. De distributie dat de meeste hoeveelheid data gebruikt is de distributie dat opgebouwd wordt door de connecties te tellen op basis van de zone ID's Dit wilt zeggen dat als er in die distributie een gat is, dat datzelfde gat zich ook in de vijf andere distributies gaat bevinden. Het probleem van deze gaten is dat er niet geweten is of die er zijn omdat een bepaalde transitie of connectie slecht is en dus niet wordt gekozen door de bezorgers of omdat er gewoon niet genoeg data is en die transitie of connectie nooit wordt waargenomen.

Een mogelijke manier om dit probleem aan te pakken is door gebruik te maken van een neuraal netwerk. Het doel van dit netwerk is om de discrete kansverdelingen van de transities te veralgemenen zodat er mogelijke gaten in de distributies opgevuld geraken dankzij informatie overdracht tussen gelijkaardige transities en zodat de kansenverdeling verder veralgemeend kan worden voor zones die nog nooit eerder gezien zijn geweest. Op deze manier kan een kansenmatrix opgesteld worden.

Als architectuur hiervoor wordt een netwerk gekozen dat elke mogelijke transitie evalueert aan de hand van een encoder en een transitie score output (of utiliteit van de transitie). Vervolgens worden alle transitie scoren gesoftmaxt tot een distributie. Het idee is dat hoe hoger de transitie score is, hoe groter de kans gaat zijn voor die transitie in de distributie. De gebruikte architectuur wordt geïllustreerd op Figuur 16. Alle linear layers hebben een ReLU activatie functie behalve de laatste die een lineaire activatie functie gebruikt om door de laatste neuron de finale utiliteit, die ook negatief mag zijn, te laten berekenen.



Figuur 16 : Architectuur gebruikt om kans distributies te veralgemenen

Als input voor elke transitie dat geëvalueerd moet worden wordt de station ID en informatie over de zone ID's gegeven. De 17 station ID's worden gecodeerd in een one-hot encoding vector van lengte 17. De zone ID's worden in een vector van lengte 5 gecodeerd. De eerste vier waarden voor deze vector worden gevonden door elk van de vier symbolen apart te coderen. Voor de letters wordt de ASCII waarde genomen en er wordt 64 van afgetrokken (de ASCII waarde van A min één). De getallen worden gewoon overgedragen tot in de input vector. Op de laatste positie staat een bit dat voorstelt of de zone een station is of niet. Als de zone een station is dan staat die bit op 1, anders op 0. De absolute verschillen tussen de overeenkomstige elementen van de vector voorstellingen van de zone ID's worden ook al op voorhand berekend en meegegeven. Op deze manier krijgen zone ID's dat op elkaar lijken gelijkaardige vector voorstellingen. Er wordt een voorbeeld gegeven:

$$vectorize('A-17.3C') = [1,17,3,3,0]$$

Hier wordt er vanuit gegaan dat de zone ID's dan ook allemaal de structuur $L-M.PR$ hebben met L en R als letters en M en P als getallen. Over het dataset is dit in maar twee gevallen niet het geval. Er zijn twee routes die hier niet aan voldoen en beide zitten in de training set, de ene bevat zone 'LOA1' en de andere bevat zone 'RTED3'. Beide zones bevatten maar een stop. Ze worden gewoon uit de zone volgordes van die routes gehaald.

Stations hebben geen zone ID, zij krijgen de vector $[0,0,0,0,1]$ toegewezen. De laatste 1 wijst erop dat het een station is.

Deze vectorisatie van zone ID's werd verkozen boven embeddings. Embeddings hebben het voordeel dat ze zelf aangeleerde voorstellingen van de zone ID's kunnen gebruiken en dus mogelijks rekening houden met relaties tussen zones die onafhankelijk zijn van de zone ID symbolen. Het nadeel van embeddings is dat embeddings van zone ID's die

nog nooit eerder gezien zijn geweest niet geleerd zouden kunnen worden, in totaal zijn er 26x100x10x26 mogelijke zone ID's. Het voordeel van vectorisaties in dit geval is dat ze direct gedefinieerd zijn voor alle mogelijke zone ID's.

Zone afstanden worden niet als inputs aan de modellen gegeven maar worden later lineair gecombineerd met de transitie kansen zoals in Canoy et al. (2022). Dit wordt op deze manier gedaan omdat het berekenen van de zone afstanden voor de hele training set ongeveer 15 minuten zou duren, wat veel te lang is aangezien de training tijd onder de 5 minuten moet blijven. Er zijn over het algemeen niet veel mogelijke input features die transitie specifiek zijn, op niveau van de zones zijn en snel voor de hele training set gegenereerd kunnen worden.

Het neuraal netwerk wordt getraind aan de hand van de CE loss. Deze loss functie is toepasselijk voor het benaderen van kansverdelingen. De kansverdeling die wordt benaderd is de teldistributie die de beste resultaten voor de eerste stap leverde. Een betere benadering van deze distributie houdt dus niet noodzakelijk in dat de TSP outputs beter zullen zijn. Daarom wordt er geprobeerd om het aantal parameters zo klein mogelijk te houden zodat er een zo goed mogelijke generalisatie gebeurd.

Het nadeel hiervan is dat er een assumptie wordt gemaakt. Over het algemeen resulteert een betere CE loss dus in betere outputs van de TSP solver maar dit is theoretisch gezien niet altijd het geval. Er kan eenvoudig een artificieel voorbeeld geconstrueerd worden waarbij een matrix een betere CE loss heeft dan een andere maar in slechtere zone volgorden resulteert.

Alternatieve manieren van learning zoals decision focussed learning of reinforcement learning (RL) lijken onpraktisch in de context van de Last Mile routing challenge. Ze zijn beter om de structuur van het probleem en de TSP solver te integreren in het leerproces maar dit komt met een kost. Doordat de TSP solver mee in het leerproces zit (voor RL in de environment) zou het trainen van een model veel te lang duren.

Een andere assumptie die hier wordt gemaakt is dat de utiliteit van elke transitie apart berekend kan worden en dat ze dus onafhankelijk zijn van elkaar. De utiliteit wordt berekend aan de hand van transitie specifieke inputs en houdt dus geen rekening met de utiliteiten van de andere mogelijke transities. Dit wordt pas later gedaan door de TSP solver.

Een voordeel van deze aanpak ten opzichte van de algemene methode (een objectief functie met artificiële beperkingen minimaliseren) is dat de penalty's zone ID's beperkingen veel beter gekozen zouden moeten worden. I plaats van een grid search over een TSP solver te doen kunnen de penalty's hier gezien worden als de gewichten van het neuraal netwerk. Deze penalty's kunnen dan ook variëren voor specifieke transities. Er hoeft dus niet één penalty gedefinieerd te worden voor elke zone ID symbool dat veranderd.

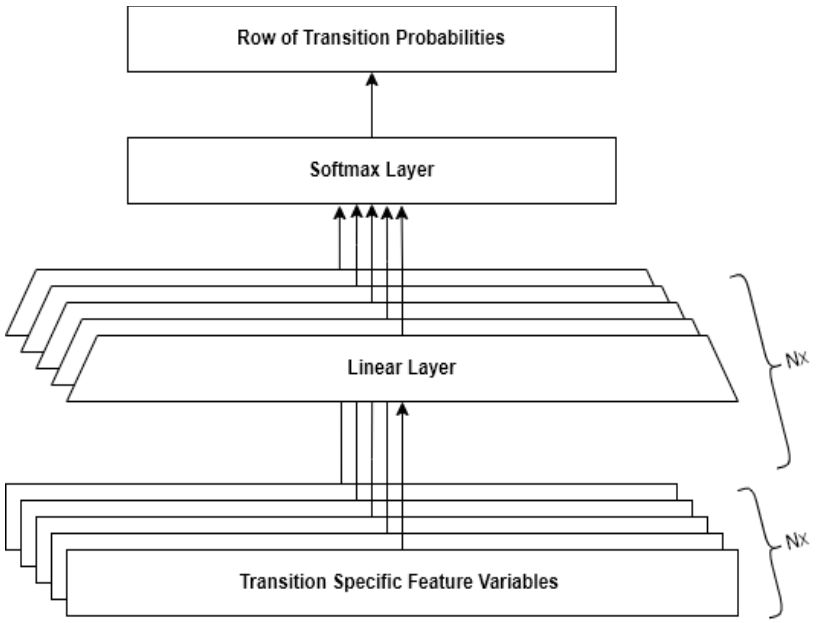
De training set bevat uiteindelijk 5,568 verschillende routes. Er gebeuren in totaal 117,190 transities en er worden 2,633,958 mogelijke transities geëvalueerd door het model.

3.6 NN voor lineaire combinaties van distributies

Een andere aanpak dat geprobeerd is geweest is geïnspireerd uit het werk van Mandi et al. (2021). Een gelijkaardige architectuur wordt gebruikt waarbij er een lineaire combinatie van Markov kansen wordt gezocht om de best mogelijke TSP inputs te krijgen.

De targets voor dit netwerk zijn in dit geval de ideale input distributies voor de TSP solver. Het zijn namelijk matrixrijen die voor elke transitie een kans van 0 hebben behalve voor de transitie die werkelijk gebeurd, die krijgt dan een kans van 1 toegewezen.

Aangezien er veel gaten aanwezig zijn in de distributies wordt bij sommige experimenten als complementaire input ook in een vector van lengte 5 de absolute verschillen tussen de overeenkomstige zone ID symbolen meegegeven. De negatieve logaritmen van de zes distributie kansen worden meegegeven in plaats van de kansen zelf omdat de output van de lineaire combinatie (de utiliteit) geen kans tussen 0 en 1 is maar een reel getal. Dit wordt gedaan zoals in Mandi et al. (2021). Op Figuur 17 wordt de architectuur geïllustreerd. Deze lijkt op die van Figuur 16, de verschillen liggen in de inputs, soms in de gebruikte targets en in het feit dat er een enkele linear layer wordt gebruikt in plaats van opeenvolgende linear layers. De gebruikte linear layer maakt gebruik van een lineaire activatie functie. Er wordt dus gewoon een lineaire combinatie van de inputs genomen.



Figuur 17 : Architectuur gebruikt voor lineaire combinaties van distributies te nemen

Aangezien er veel symmetrische inputs zijn worden de dezelfde experimenten ook uitgevoerd maar door een symmetrische distributie als target te gebruiken. Hierbij zijn de kansen van alle transities gelijk aan 0 behalve voor de twee buurzones waaruit de transitie vertrekt, zij krijgen alle twee kansen van 0.5 toegewezen.

Dit netwerk wordt ook getraind aan de hand van CE loss. Deze keer worden de ideale TSP inputs gebruikt om het model te trainen. Deze targets zijn namelijk niet noodzakelijk realistisch: de transitie die moet gebeuren heeft niet noodzakelijk een kans van één en de transities die niet moeten gebeuren hebben niet noodzakelijk een kans van nul.

Net als in Mandi et al. (2021) wordt hier ook getest door verschillende linear layers te gebruiken voor dezelfde source node. Dit houdt in dat wanneer een transitie vertrekt vanuit een bepaalde zone (of node in het algemeen) dat er dan een specifieke layer wordt gebruikt voor die transitie te evalueren. Voor de Last Mile routing challenge is het onduidelijk wat een ‘source’ (of bron) precies zou moeten zijn. Er kan namelijk gekozen worden om een bron te definiëren aan de hand van een zone ID of om ze te definiëren door gebruik te maken van de unieke ID’s. Beide mogelijkheden worden getest.

Doordat er in deze architectuur verschillende layers voor verschillende bronnen worden gebruikt kan het model op verschillende manieren getraind worden. De layers kunnen collectief getraind worden door alle training data in een keer te geven en het model te laten stoppen wanneer de minimale validation loss is bereikt. In dit geval zou de validation loss het gemiddelde voorstellen van de validation losses van alle layers. Een andere manier is om elke layer individueel te trainen met de data dat die specifieke layer gebruikt. Op deze manier kunnen de validation losses van elke layer individueel geminimaliseerd worden en kan elke layer voor een verschillende hoeveelheid tijd of epochs trainen.

Voor beide definities van wat een ‘source’ is, is het onduidelijk wat er gedaan moet worden wanneer de source nog nooit eerder gezien is geweest. Dan is er ook geen gespecialiseerde linear layer dat daarvoor gebruikt kan worden. Hiervoor wordt er een extra ‘default’ layer gemaakt. Dit is de layer dat gebruikt wordt in het geval dat de source nog nooit eerder gezien is geweest. Het trainen van de default layer is ook onduidelijk aangezien er geen data voor is. Daarom wordt er aan de hand van de definitie van wat een bron is gekeken hoeveel procent van de transities in de validatie set de default layer gaan gebruiken. Dit percentage wordt dan gebruikt om een deel van de training set te verdubbelen, bij de verdubbelde instanties wordt er alsof gedaan dat de bron niet gekend is. Dit deel van de training set gaat dan gebruikt worden om de default layer te trainen.

Hoofdstuk 4: Numerieke resultaten

In dit hoofdstuk worden de numerieke resultaten van de experimenten besproken en geïnterpreteerd. De experimenten werden gerund op een Intel Core i7-8750H CPU @2.20Ghz, 2201 Mhz, 6 Cores en 16 RAM gerund. De experimenten worden gerund in Jupyter Notebooks met de Python versie 3.9.2 en Tensorflow versie 2.8.0. Voor het initialiseren van netwerken wordt altijd dezelfde random seed 759079 gebruikt. Dit is belangrijk aangezien neurale netten sensibel zijn voor hun initialisatie. Voor het trainen van netwerken worden een batch size van 32 gebruikt, een patience van 1 voor de Early Stopping Callback, optimizer adam met een learning rate van 0.001 en een training tijd van max 5 minuten tenzij anders vermeld. Als validatie set worden de transitie gebruikt van de 544 validatie routes gebruikt. Als TSP solver wordt de Google OR Tools TSP solver behouden. Zowel in stap een als twee wordt de First Solution Heuristic gebruikt. In stap een met een tijdslimiet van 1 second en in stap twee van 120 seconden. Er is altijd een oplossing gevonden geweest binnen deze tijdslimieten. De tweede stap duurt ongeveer 40 minuten, daarom wordt de stop score niet voor alle modellen berekend maar wel voor de betere in stap een of de interessantste. De langste zone volgorde bevat verschillende 48 zones. Transities worden dus met zero-pads tot een lengte van 50 verlengd. Transities van een zone naar zichzelf worden ook gepad. Wanneer een neuraal netwerk wordt gebruikt om kansverdelingen te genereren wordt achteraf een smoothing van 10^{-8} gebruikt om te voorkomen dat het logaritme van 0 wordt genomen. De gebruikte gewichten voor de loss functie zijn 3, 2 en 1 voor hoge, gemiddelde en lage kwaliteit transities. De kwaliteit van een transitie wordt bepaald door de kwaliteit van de route waarin dat die transitie gebeurd.

Gebruikte metrieken om de methodes en modellen te evalueren zijn voor stap een: zone score, aan de hand van de gegeven score functie voor de challenge met zone afstanden als afstandsfunctie, en Arc Difference (AD), dat het verschil weergeeft tussen de set van transities van de voorgestelde volgorde en de set van transities van de waargenomen volgorde. De AD van een volgorde wordt uitgedrukt in een percentage, voor de hele validatie set wordt de gemiddelde AD gebruikt. Flipped AD wordt gebruikt als naam voor de nieuwe AD nadat het referentie mechanisme is gebruikt geweest om bepaalde volgorden te inverteren. Flipped AD is dus de AD na postprocessing.

4.1 Theoretische experimenten

De eerste set van experimenten waren om benchmarks te hebben aan de hand van traditionele methoden en om te testen of de tweede stap ook in staat was om een bijna perfecte stop score te produceren als de perfecte zone volgorden worden gegeven.

Als het probleem wordt aangepakt door gewoon de reistijden te minimaliseren wordt een stop score van 0.091 behaald. Als de zones als clusters worden beschouwd en

elke keer dat er van cluster veranderd wordt, wordt er een penalty van 10 bijgerekend op de genormaliseerde reistijden, dan wordt de stop score 0.061.

Indien de zone volgorde gegeven is en er wordt een clustered ATSP aanpak gevolgd waarbij de penalty altijd van 10 is maar er is geen penalty bij zone transities als die transitie in de zone volgorde gebeurd dan is de score 0.037. Indien er een penalty van 5 wordt toegepast ook al wordt er een transitie gedaan die ook in de zone volgorde is dan wordt de stop score 0.032. Dit promoveert meer dat zones apart behandeld worden. Tot slot, als de zone volgorde wordt gegeven en de penalty's van de tweede stap van het 2-staps algoritme worden toegepast dan kan er een stop score van 0.0079 bereikt worden. Indien de richting waarin de volgorde gerespecteerd moet worden door de TSP solver wordt gekozen, dan wordt deze score 0.0190. Dit bewijst dat de richting van de voorgestelde volgorde van belang is.

Een laatste experiment dat nog bij de theoretische experimenten hoort is om te testen hoeveel fouten het referentie mechanisme maakt indien er perfecte zone volgorden worden gepresenteerd door de eerste stap. Indien er perfecte zone volgorden worden gegenereerd in de eerste stap zijn de zone score en de AD natuurlijk alle twee 0. Indien de geïnverteerde perfecte zone volgorden worden voorgesteld dan is de zone score 0 maar de AD 100%. Alle transities komen namelijk niet voor in de waargenomen volgorde, dit geldt voor elke route. Indien de richtingen gekozen worden door het referentie mechanisme dan wordt een zone score van 0 en een AD van 6.06 verkregen. Deze volgorden resulteren dan in een stop score van 0.0092 in stap twee, dus niet veel slechter dan 0.0079.

4.2 Experimenten op manieren van tellen

Markov counting kan in dit dataset op verschillende manieren gebeuren. In de experimenten worden routes met hoge, gemiddelde en lage kwaliteit respectievelijk de gewichten 3, 2 en 1 gegeven. Er wordt geteld op drie verschillende niveaus en op twee verschillende manieren. De drie niveaus zijn globaal, stedelijk en station. Bij globaal worden zone ID's geteld over de hele dataset, bij stedelijk worden alleen de routes gebruikt die bij dezelfde stedelijke ruimte horen en op het niveau van een station worden allen routes gebruikt die vanuit hetzelfde station vertrekken. De twee manieren van tellen zijn transities en connecties. Hiernaar wordt ook gerefereerd als asymmetrisch en symmetrisch respectievelijk omdat connecties gezien kunnen worden als symmetrische transities. Op Tabel 1 worden de resultaten getoond van alle telmethodes en op Tabel 2 worden de resultaten getoond van diezelfde telmethodes gecombineerd met de afstandskansen. Het resultaat dat wordt bekomen door alleen de afstandskansen te gebruiken wordt ook getoond. De lineaire combinatie dat is gebruikt voor de kansenmatrices is dezelfde als die dat door SOP werd gevonden in Canoy et al. (2022), namelijk 2.33 en 1.58 voor transitie-of connectie kansen en afstandskansen respectievelijk.

Counting method	zone score	AD	Flipped AD
transitions global	0.0372	41.4	29.28
transitions city	0.0512	33.69	34.94
transitions station	0.0586	**30.31**	37.01
connections global	**0.0352**	59.41	**26.95**
connections city	0.0472	62.22	32.99
connections station	0.0584	64.16	40.45

Tabel 1 : Resultaten experimenten op verschillende manieren van tellen

Wanneer de connecties geteld worden in plaats van transities wordt er een betere zone score behaald. Dit is waarschijnlijk te danken aan de toename in data. Het nadeel hiervan is ook duidelijk, de AD's van de connecties zijn veel slechter dan die van de transities. Dit wordt vervolgens opgelost door het referentie mechanisme dat bepaalde volgorden gaat inverteren en terug voor betere AD's gaat zorgen, de Flipped AD's.

Van de asymmetrische tellen is het station niveau de beste (het heeft de lagere AD). Dit komt omdat het beter de lokale preferenties van de bezorgers leert. Dezelfde zone ID's die in andere stedelijke ruimten zijn creëren hier geen interferenties. De globale transities hebben een iets betere Flipped AD. Zij bevatten meer informatie over de zone ID beperkingen maar maken blijkbaar nog veel fouten op de richtingen. Bij transitie kansen zou het niet de bedoeling moeten zijn om nog op het referentie mechanisme moeten rekenen aangezien de informatie over de richtingen intrinsiek in de distributies aanwezig zou moeten zijn aangezien de tellen asymmetrisch zijn.

Voor de symmetrische tellen zijn de globale tellen de beste. Dit komt waarschijnlijk omdat ze in staat zijn om de zone ID beperkingen beter te modelleren zoals de 'verschil van één' beperking. Het verschil tussen stedelijke connecties en station connecties is ook duidelijk te zien bij hun Flipped AD's. Stedelijke connecties maken gebruik van meer data dan station connecties zonder interferenties te maken tussen unieke ID's.

Counting method with distances	zone score	AD	Flipped AD
distances	0.0642	70.7	64.89
transitions global	0.0285	39.26	23.92
transitions city	0.0324	28.82	26.03
transitions station	0.0344	**24.05**	28.13
connections global	**0.0281**	54.58	**22.77**
connections city	0.034	59.54	26.09
connections station	0.0359	59.9	30.37

Tabel 2 : Resultaten experimenten op verschillende manieren van tellen met afstanden

Het gebruik van afstanden verbeterd elke distributie. Dit komt waarschijnlijk omdat er niet veel data is om te tellen en wanneer er een voorspelling gedaan moet worden voor een transitie die nog nooit eerder gezien is geweest kunnen de afstanden hierbij helpen. De beste tellen blijven de globale connecties en de station transities, die alle twee op

verschillende patronen focussen (globale connecties op zone ID patronen en station transities op lokale voorkeuren). De meeste relaties tussen de resultaten blijven dezelfde.

Het is merkwaardig om te zien dat stedelijke transitie tellen slechtere resultaten geven dan de station transitie tellen op basis van hun AD maar in een betere zone score resulteren. Dit komt waarschijnlijk omdat richtingen sterk afhankelijk zijn van de set van zones die bezocht moeten worden en dus de route in haar geheel. Wanneer eenzelfde geografische zone bereikt wordt door twee verschillende stations dan zijn de andere zones in de routes waarin ze bereikt wordt heel verschillend. Dit zorgt ervoor dat de tellen die vanuit twee verschillende stations komen met elkaar interfereren op vlak van de richtingen. Bij de stedelijke en station connectie tellen is dit niet het geval want de richtingen worden bepaald door het referentie mechanisme waardoor een beter zone score dan ook in een betere Flipped AD resulteert. Er wordt dus gewoon meer data gebruikt in de stedelijke connectie tellen dan in de station connectie tellen zonder de richtingen te laten interfereren want ze zijn symmetrisch. Stedelijke tellen bevatten de meeste hoeveelheid data zonder interferenties te veroorzaken tussen geografisch verschillende zones met dezelfde zone ID's.

De stop score van de globale connectie distributie werd berekend en die resulteerde in 0.02762, wat al beter is dan de oorspronkelijke Markov counting met 0.0465.

4.3 Lineaire combinaties van tellen en distributies

Er wordt getest om lineaire combinaties te nemen van de 6 verschillende distributies. Er kan ook een stap achteruit gezet worden en een lineaire combinatie van de 6 tellen genomen worden tot één enkele tel. De CE loss op zowel de training set als de validatie set worden ook getoond. Zowel de symmetrische (Tabel 3) als asymmetrische (Tabel 4) ideale kansverdelingen worden als targets gebruikt voor deze experimenten. Bij de linear layer met 6 kansverdelingen wordt er ook getest om de verschillen van de zone ID's meegegeven. Vervolgens wordt er getest met alleen 2 van de 6 kansverdelingen of tellen te werken om te kijken of er overfitting gebeurde. Voor de connectie targets worden de globale en stedelijke connectie kansen/tellen gebruikt terwijl voor de transitie targets worden de globale connectie kansen/tellen gebruikt samen met de station transitie kansen/tellen. In alle resultaten worden de afstanden al meegerekend.

Connection targets (Linear layer with ...)	zone score	AD	Flipped AD	CE loss	Validation CE loss
6 probabilities	0.0325	82.07	22.83	4.5535	6.015
6 counts	0.0305	**52.81**	23.34	**3.9897**	**5.8941**
6 probabilities and zone ID differences	0.0324	82.83	23.02	4.2389	6.0195
2 probabilities	0.0299	55.5	22.31	4.4805	5.997
2 counts	**0.0295**	56.56	**22.08**	4.3185	6.0408

Tabel 3 : Linear Layer experimenten met ideale connectie kansverdelingen

Een lineaire combinatie gebruiken van de tellen blijkt beter te zijn voor de AD, CE loss en validatie CE loss. De Flipped AD is wel niet beter dan die van de globale connectie kansen. Alleen wanneer er maar twee tellen worden gebruikt resulteert dit in een iets betere Flipped AD dan de globale connectie kansen.

Transition targets (Linear layer with ...)	zone score	AD	Flipped AD	CE loss	Validation CE loss
6 probabilities	**0.0304**	**21.23**	**23.49**	2.9646	4.866
6 counts	0.0338	22.61	26.32	2.9027	**4.5606**
6 probabilities and zone ID differences	0.0305	21.31	23.93	3.1053	4.8384
2 probabilities	0.0326	22.99	26	3.0822	5.247
2 counts	0.0347	23.75	27.13	**2.5032**	4.6002

Tabel 4 : Linear layer experimenten met ideale transitie kansverdelingen

De lineaire combinatie van 6 kansendistributie blijkt de beste optie te zijn wanneer de transitie targets worden gebruikt. Ze halen een AD dan bet is dan de vorige globale connectie kansen. Wanneer er maar twee tellen worden gebruikt wordt er een betere CE loss bereikt, wat dus blijkt te overfitten met minder inputs. De beste validatie CE loss wordt gehaald door de 6 tellen maar dit resulteert niet in betere resultaten.

In beide gevallen heeft het expliciet toevoegen van de zone ID verschillen geen impact gehad. Het idee was dat ze de verantwoordelijkheid en functie van de globale connectie kansen zouden overnemen wanneer er geen data zou zijn geweest voor een bepaalde transitie.

Over het algemeen heeft deze set van experimenten geen grote verbeteringen gebracht aan het beste model, namelijk de globale connectie distributies.

De stop scores voor stap twee zijn berekend geweest voor de linear layers met de 6 kansverdelingen. Voor de connectie targets was de stop score 0.029 terwijl de transitie targets een score van 0.0322 behaalden.

4.4 Lineaire combinaties voor elke source

Zoals in Mandi et al. (2021) wordt er ook geëxperimenteerd met verschillende linear layers voor verschillende source zones. In deze context worden sources op twee verschillende manieren gedefinieerd: op basis van de zone ID's en op basis van unieke ID's. Zowel transitie als connectie targets worden gebruikt en de linear layers worden zowel collectief als individueel getraind. Modellen worden met en zonder afstandskansen getest. Individuele training is gedaan met batch sizes van 1, 1000 epochs per linear layer, een learning rate van 0.01 en een patience van 10. De inputs van de linear layers zijn de 6 kansendistributies. In Tabel 5 en Tabel 6 worden respectievelijk de resultaten getoond voor het collectief en individueel trainen.

Source is zone ID
Collective training

Connection targets	zone score	AD	Flipped AD	CE loss	Validation CE loss
Linear layers	0.0388	76.3	33.17	3.4329	5.4318
Linear layers with distances	**0.0356**	76.0	**29.48**		
Transition Targets					
Linear layers	0.0456	31.8	41.15	2.3101	5.5233
Linear layers with distances	0.0409	**30.6**	38.21		

Tabel 5 : Collectieve training van linear layers waarbij source bepaald wordt door zone ID

Source is zone ID
Individual training

Connection targets	zone score	AD	Flipped AD	CE loss	Validation CE loss
Linear layers	0.0371	78.8	30.76		5.3751
Linear layers with distances	**0.0333**	79.3	**27.10**		
Transition Targets					
Linear layers	0.0398	26.08	34.03		5.4638
Linear layers with distances	0.037	**24.91**	32.73		

Tabel 6 : Individuele training van linear layers waarbij source bepaald wordt door zone ID

Met deze source configuratie zijn er in totaal 8908 mogelijke sources waarvan een de default layer voorstelt. De default layer wordt gebruikt wanneer er een zone ID als source wordt tegengekomen die niet in de training set voorkomt is. In totaal zijn er in de validatie set 78 transities die gebruik moeten maken van de validatie set. Dit komt overeen met ongeveer 0.7% procent van de validatie set. Er wordt dus 1% van de training set gedupliceerd om de default layer te trainen. Er zijn gemiddeld 13 transities per layer om te kunnen trainen. Voor de training set zijn er gemiddeld maar 2.3 transities per layer. Uit de resultaten kan gezien worden dat individuele training inderdaad de validatie CE loss kan verbeteren. De verbeteringen zijn in dit geval niet heel groot maar de effecten ervan kunnen wel gemerkt worden bij de zone scores, AD's en Flipped AD's die voor elk model betere resultaten geven.

Training tijden voor de individuele modellen waren over 5 minuten. Ze waren 13.62h en 46.30h voor connectie en transitie targets respectievelijk. Er werd geen custom training loop geschreven. De fit functie van tensorflow werd opgeroepen voor elke layer met de

overeenkomstige data. Alleen de layers die nodig waren voor de validatie set te evalueren zijn getraind geweest bij de individuele trainingen. Hoe de andere layers getraind kunnen worden zonder validatie data is onduidelijk. Een mogelijke aanpak zou kunnen zijn om ze voor het gemiddelde aantal epochs te trainen dat de andere layers nodig hebben gehad.

De stop scores zijn berekend geweest voor elk model en kunnen gezien worden op Tabel 7. Hierop kan gezien worden dat individuele training ook in stap twee een invloed heeft.

Stop scores	Connection targets	Transition targets
Collective training	0.0326	0.0418
Individual training	**0.0321**	0.0387

Tabel 7 : Stop scores voor linear layer model met source bepaald door zone ID

Deze modellen behalen over het algemeen slechtere resultaten dan de vorige modellen. Dit kan te wijten zijn aan te weinig data om aparte linear layers te trainen. Er is vooral te weinig validatie data.

Collectieve training wordt ook getest door een bron op basis van unieke ID's te definiëren. Dit resulteert in 35894 verschillende linear layers, waaronder de default layer. De default layer behandelt 1085 transities van de validatie set dat overeenkomt met 9.5% van de validatie set. Er wordt dus 10% van de training set gedupliceerd om de default layer te trainen. Er zijn gemiddeld 3.2 transities om elke layer te trainen en 1.4 transities in de validatie set van elke layer. In Tabel 8 en Tabel 9 worden respectievelijk de resultaten getoond voor het collectief en individueel trainen. Op Tabel 10 kunnen de resultaten hiervan in stap twee gezien worden.

Source is unique ID
Collective training

Connection targets	zone score	AD	Flipped AD	CE loss	Validation CE loss
Linear layers	0.0471	71.69	41.92	3.269	6.069
Linear layers with distances	**0.0396**	70.47	**36.72**		
Transition Targets					
Linear layers	0.059	35.3	45.55	2.162	6.216
Linear layers with distances	0.0523	**33.85**	44.71		

Tabel 8 : Collectieve training van linear layers waarbij source bepaald wordt door unieke ID

Source is unique ID
Individual training

Connection targets	zone score	AD	Flipped AD	CE loss	Validation CE loss
Linear layers	0.0478	69.18	39.49		5.049
Linear layers with distances	**0.0398**	68.62	**36.33**		
Transition Targets					
Linear layers	0.0572	34.91	45.13		5.118
Linear layers with distances	0.0505	**33.2**	44.64		

Tabel 9 : Individuele training van linear layers waarbij source bepaald wordt door unieke ID

Stop scores	Connection targets	Transition targets
Collective training	**0.0374**	0.0498
Individual training	0.0384	0.049

Tabel 10 : Stop scores voor linear layer model met source bepaald door unieke ID

De modellen waarbij de linear layers individueel getraind worden halen weer betere validatie losses. Het trainen van deze modellen duurde 63.46h en 96.74h voor de connectie en transitie targets respectievelijk. Dit is veel meer dan bij de andere definitie van de bronnen. Dit komt omdat de fit functie van tensorflow veel vaker wordt opgeroepen doordat er veel meer linear layers zijn. Deze modellen behalen slechtere resultaten dan die met de bron als zone ID. Ze overfitten meer en hebben nog minder data om te trainen en te valideren.

4.5 Veralgemenen van distributies

4.5.1 veralgemenen van Markoviaanse tellen en distributies

Uit de experimenten met de linear layers blijkt dat ze niet veel verbetering brengen aan de initiële distributies. Een mogelijke verklaring hiervoor zou kunnen zijn dat de initiële distributies niet goed genoeg zijn en dus ook al worden er lineaire combinaties ervan gezocht blijft het resultaat niet goed genoeg. Dit zou vooral te wijten zijn aan het feit dat de distributies elkaar niet complementeren bij gebrek aan data. Zoals eerder al besproken is geweest, als er een ‘gat’ (geen data voor een bepaalde transitie) is in de

globale connectie distributie dan gaat dat gat zich ook in alle andere distributies bevinden. De enige distributie dat dat gat momenteel opvult is de afstandsdistributie.

Om in het Markov counting framewerk te blijven wordt de beste initiële distributie benaderd aan de hand van een neuraal netwerk. Het idee hierachter is om tel informatie van sommige transities te laten doorvloeien naar gelijkaardige transities. Als target distributie wordt de globale connectie distributie gebruikt. Aangezien in dit geval, een betere validatie loss betekent dat de initiële distributie beter benaderd wordt, worden de resultaten op stap een gebruikt om te beslissen wat de beste design is in termen van aantal layers en neuronen. De hoogste layer heeft 32 neuronen die verbonden staan aan de laatste utiliteitsneuron met een lineaire activatiefunctie. Deze layer is direct verantwoordelijk voor de utiliteit van een transitie. Vervolgens worden er layers toegevoegd met als aantal neuronen toenemende machten van 2. De inputs dat aan het netwerk worden gegeven zijn one-hot encodings van de station ID's en de vectorisaties van de twee zone ID's. Op Tabel 11 worden de resultaten hiervan getoond.

layers design	zone score	AD	Flipped AD	#parameters
32	0.0288	**56.45**	20.57	961
64-32	0.0277	57.15	20.55	3,969
128-64-32	**0.0273**	58.44	**19.91**	14,081
256-128-64-32	0.0278	56.57	20.06	50,689

Tabel 11 : Resultaten op globale connectie targets afhankelijk van het aantal parameters

Uit de resultaten kan er vastgesteld worden dat de 128-64-32 design de in beste zone score en Flipped AD resulteert. De behaalde stop score van dit model is 0.0261. Dit is een verbetering is ten opzichte van de stop score van de initiële globale connectie distributie (0.02762).

Hetzelfde design is ook gebruikt geweest om een combinatie te benaderen van de globale connectie tellen en van de stedelijke connectie tellen. Dit wordt gedaan omdat ze bij de experimenten op lineaire combinaties van tellen en distributies goede resultaten leverden. De tellen worden eerst opgeteld met gewichten 1:1 en worden vervolgens genormaliseerd door te delen door de som van de rij elementen. Dit is anders dan hoe het door de linear layer werd gedaan. In de linear layer werden de gewichten en bias zelf gezocht door het neuraal netwerk en de normalisatie gebeurde door een softmax layer. Op Tabel 12 worden de resultaten hiervan getoond.

Generalisation of global + city connection counts	zone score	AD	Flipped AD
Neural Net	0.0270	**59.2**	20.18
Neural Net with distances	**0.0270**	59.4	**19.80**

Tabel 12 : Resultaten van generalisatie van globale en stedelijke tellen

De resultaten zijn iets beter dan de globale connectie kansen alleen te generaliseren. De stop score die hierdoor behaald werd is 0.0257, dus ook iets beter. Het is ook opmerkelijk om te zien dat afstanden hier voor maar een heel kleine verbetering zorgen.

Een andere methode die getest is geweest is door dit model te gebruiken met de station transitie kansen. De station transitie distributie wordt hierbij achteraf opgeteld samen met de afstanden distributies. Het gewicht voor de station transitie distributie is gebruikt is ook 2.33. Resultaten worden getoond op Tabel 13.

	zone score	AD	Flipped AD	stop score
Generalised global connections + station transitions + distances	0.0253	18.08	20.16	0.027

Tabel 13 : Resultaten van generalisatie van globale connecties, station transities en afstanden

Het model heeft de beste zone score en AD tot nu toe. Dit komt omdat de gegeneraliseerde globale connecties een goede basis vormen voor de zone ID beperkingen te respecteren terwijl de station transitie kansen goede lokale preferenties kunnen modelleren. Een combinatie hiervan geeft dus ook goede resultaten. De stop score is minder goed dan verwacht dit zou kunnen komen doordat de station transitie kansen ook nog eens informatie bevatten over de zone ID beperkingen die al gemodelleerd zijn. Di kan voor interferenties zorgen tussen de twee distributies en verstoringen brengen op vlak van de zone ID beperkingen.

4.5.2 Veralgemenen van ideale connectie en transitie kansen

Dezelfde aanpak kan gedaan worden om de ideale connectie en transitie kansen te benaderen. Dit zou algemener moeten zijn en loshangen van de Markoviaanse tellen of distributies. Aangezien de targets nu ideale distributies zijn kan er wel terug met de validatie loss gewerkt worden. Hier wordt gebruik van gemaakt om een betere hyperparameter search voor de design toe te passen.

Het algoritme dat hiervoor wordt gebruikt is Hyperband van Keras. De modellen worden gemaakt met minimaal 1 layer en maximaal 5. Per layer zijn er minimaal 32 neuronen en maximaal 128. De opties worden doorzocht met stappen van 32 neuronen. Er worden maximaal 10 epochs uitgevoerd zodat de training onder 5 minuten blijft en er wordt ene factor van 5 gebruikt voor Hyperband samen met een Early Stopping Callback met patience van 1. De resultaten van de modellen die geen gebruik maken van de 6 initiële distributies als input worden getoond op Tabel 14 en de resultaten van de modellen die er wel gebruik van maken worden getoond op Tabel 15.

Targets	layers design	zone score	AD	Flipped AD	CE loss	Validation CE loss	stop score
connections	96-96-96-32	**0.0275**	58.8	**20.43**	3.584625	4.4355	**0.0265**
transitions	96-64-32-96	0.0279	**55.24**	21.17	3.771144	4.6317	0.027

Tabel 14 : Resultaten van generalisatie van ideale connectie en transitie distributies met hyperparameter tuning

De zone scores van beide modellen zijn beter dan die van de globale connectie kansen. De AD van het model dat getraind is op de transitie kansen is merkwaardig hoog. Die wordt verbeterd door het referentie mechanisme maar dat zou niet de bedoeling mogen zijn, het model zou zelf de richtingen moeten aanleren. Dit kan te maken hebben met het feit dat de enige asymmetrische inputs de zone ID's zijn en dat

het voor het model moeilijk is om dit soort preferenties te leren en het eenvoudiger is om ongeveer symmetrische outputs te geven. Beide modellen produceren goede stop scores.

Targets	layers design	zone score	AD	Flipped AD	CE loss	Validation CE loss
connections	32	**0.037**	46.58	31.42	2.9524	4.9032
transitions	128-32-128-128-96	0.0389	**25.79**	**30.47**	3.3906	4.1976

Tabel 15 : Resultaten van generalisatie van ideale connectie en transitie distributies met hyperparameter tuning met 6 initiële distributies als inputs

Het toevoegen van de 6 initiële distributies maakt de modellen slechter. De AD van het model dat getraind is op de transitie kansen in wel verbeterd. Dat model haalt nu ook een betere validatie loss maar dat help blijkbaar niet. Het model getraind op de connectie kansen heeft een heel simpel design met één layer van 32 neuronen. Het model overfit duidelijk op de training data omdat de validatie loss verslechterd is .

4.6 Conclusie en reflectie uit experimenten

De theoretische experimenten zijn veelbelovend. Er kan namelijk een stop score van 0.0079 behaald worden indien de zone volgorde perfect gekend is. Dit wilt niet zeggen dat het ook mogelijk is om deze waarde te bereiken maar is wel een goede motivatie om de eerste stap te proberen te verbeteren. Het referentie mechanisme heeft ook goede theoretische resultaten behaald met een AD verslechtering van 0 naar 6 voor perfecte zone volgorden, wat zich vertaald naar een stop score verslechtering van 0.0079 naar 0.0092. Het referentie mechanisme heeft bewezen om heel nuttig te zijn bij de andere experimenten. Dit wilt niet zeggen dat het mogelijk is om de perfecte connectie volgorden te construeren zonder enige informatie over de richtingen te gebruiken.

De zes initiële kansendistributies die ontstaan door anders te tellen bevatten ook veel informatie op verschillende niveaus. De globale connectie kansen, die uit de grootste hoeveelheid data worden gebouwd kunnen een stop score van 0.0276 met behulp van het referentie mechanisme.

Lineaire combinaties nemen van de distributies en van manieren van tellen heeft niet veel geholpen. Dit komt waarschijnlijk omdat de distributies en de tellen elkaar niet echt kunnen complementeren. De data sets waarmee ze worden opgesteld zijn subsets van elkaar. Verschillende lineaire combinaties gaan nemen afhankelijk van wat de bron van de transitie is heeft ook niet geholpen. Daar is te weinig data voor en het probleem dat de distributies elkaar niet complementeren blijft.

Het benaderen van de globale connectie distributies heeft goede resultaten geleverd. Dit komt waarschijnlijk omdat het neuraal netwerk de echte onderliggende distributie goed kan benaderen. Een nadeel hiervan is dat het een heel indirecte aanpak is en

dat het moeilijk is om de hyperparameters van het netwerk te tunen. De beste stop score dat met deze methode bereikt is geweest is van 0.0257.

Het benaderen van de ideale transitie en connectie kansen heeft ook tot goede resultaten geleidt. Deze aanpak is veel directer omdat de ideale TSP inputs worden benaderd. Een nadeel hiervan is dat ze niet noodzakelijk de reële onderliggende distributies voorstellen. Het toevoegen van de zes initiële kansen zorgde voor interferenties en verslechterde de resultaten. Alleen de zone ID's en de station ID bevatten blijkbaar niet genoeg informatie om op deze manier richtingen te bepalen.

De resultaten die getoond worden voor de metrieken zijn ook maar gemiddelde warden op de 544 test routes. De distributies van de metrieken zouden kunnen verschillen. Er zouden in de eerste stap slechte uitschieters kunnen zijn die dan in stap twee voor nog grotere fouten zorgen.

De netwerken die worden voorgesteld door Hyperband zijn groter dan verwacht. De inputs van de netwerken zijn de zone ID's en de station ID. Om de zone ID beperkingen te simuleren en de absolute verschil tussen de overeenkomstige symbolen te nemen is maar een linear layer met ReLU activatie functie van 10 neuronen nodig die dan verbonden is met de finale utiliteitsneuron voor een transitie. Het feit dat de voorgestelde designs hier zo hard van afwijken impliceert dat er meer is dan de algemene zone ID beperkingen die besproken zijn geweest in de literatuur studie.

De modellen kunnen de zone ID beperkingen uit de literatuur studie simuleren en gebruiken waarschijnlijk ook nauwkeurigere penalty's, aangezien ze geleerd worden door een neurale net in plaats van, voor een gelimiteerd aantal beperkingen, gezocht te zijn aan de hand van grid searches op een LKH solver of een genetisch algoritme.

De stop scores liggen ook in het bereik van de winnende teams maar wel op een andere test set. De 544 test set die gebruikt is geweest is mogelijks moeilijker dan de 1107 test set die door twee van de drie winnende teams werd gebruikt. Dit is omdat er in de 544 test set routes zitten die meerdere keren doorheen dezelfde zones gaan, wat bij de 1107 test set niet het geval is. Een ander aspect is wel dat door de 544 test set te gebruiken er meer training data ter beschikking is, wat essentieel is voor modellen in het Markov counting framewerk.

Hoofdstuk 5: Suggesties voor toekomstig onderzoek

5.1 Input variabelen

Uit het experiment waarbij ideale transitiekansen gegeneraliseerd worden blijkt dat het niet lukt om de juiste richtingen te bepalen. Dit zou te maken kunnen hebben met de gebruikte inputs die alleen informatie bevatten over de zone ID's en de station ID. In het werk van Guo et al. (2023) word het probleem van het bepalen van de richtingen aangepakt in een postprocessing fase door ervoor te zorgen dat de grotere pakketten en pakketten die een langere levertijd hebben eerst worden geleverd. Informatie over de grootte, aantal en levertijden van pakketten zou samengevat kunnen worden aan de hand van drie gemiddeldes voor elke zone. Deze gemiddeldes kunnen dan relatief snel berekend worden en aan het neuraal netwerk gegeven worden. Het idee hierachter is dat het netwerk een grotere utiliteit zou geven aan zones met grotere gemiddeldes als het gaat over een transitie vanuit het station. Vervolgens zouden transities tussen zones met gelijkaardige gemiddeldes ook een grotere utiliteit moeten krijgen. Dit kan ervoor zorgen dat richtingen door de TSP solver bepaald worden en niet in een postprocessing stap, wat dus geen assumptie maakt over de onafhankelijkheid van het bepalen van connecties en richtingen. Het is natuurlijk niet vanzelfsprekend dat de informatie over pakketten genoeg is om de richtingen te bepalen.

De designs van de netwerken toonden aan dat er duidelijk meer is dan de algemene zone ID beperkingen. Om dus betere scores te verkrijgen is het van belang om de lokale voorkeuren beter te modelleren, dit werd ook bewezen door het experiment waarbij de generalisatie van de globale connecties werd gebruikt met de station transities. De representatie van de zone ID's introduceert een bias, namelijk dat de symbolen waaruit de zones bestaan geordend zijn. Deze bias is niet verkeerd en heeft de basis gevormd voor alle winnende algoritmen maar het zou het model wel kunnen tegenhouden om preferenties te leren die niet aan de algemene zone ID beperkingen voldoen. Een simpel voorbeeld hiervan zijn de stations. Ze krijgen een representatie van [0,0,0,0,1] en hun transities kunnen niet zomaar geëvalueerd worden aan de hand van zone ID beperkingen. Een betere representatie voor de zone ID's zou met embeddings op level van de symbolen kunnen zijn. In plaats van een orde op te leggen van de symbolen kunnen latente representaties ervan geleerd worden die dan, gekoppeld met de andere symbolen, de lokale preferenties beter kunnen modelleren.

5.2 Referentie mechanisme

Het postprocessing referentie mechanisme heeft voor goede resultaten gezorgd maar het is gebaseerd op de assumptie dat connectie volgorden en richtingen onafhankelijk bepaald kunnen worden. Deze veronderstelling zou mogelijks de prestatie kunnen

limiteren. De beste optie om deze veronderstelling niet te maken is om richtingen direct, samen met de zone volgorde, door de TSP solver te laten bepalen en niet in een postprocessing stap waarbij de volgorde alleen nog geflipt kan worden. Voorrang beperkingen hebben zowel in het werk van Cook et al. (2021) als in de experimenten bewezen dat ze enorm handig kunnen zijn om richtingen te bepalen. Het probleem van deze voorrang beperkingen is dat ze niet transitie specifiek zijn. Ze kunnen dus niet uitgedrukt worden in een afstanden matrix, laat staan als input geleverd worden aan een ASU-DNN. De eenvoudigste manier om ze te integreren is dus aan de hand van een aparte penalty functie zoals in Cook et al. (2021). De huidige TSP solver ondersteunt geen bijkomende penalty functies, die zou dus veranderd moeten worden.

5.3 Generatieve modellen

In de experimenten werden modellen getest die gebaseerd zijn op de veronderstelling dat de utiliteit van een individuele transitie onafhankelijk bepaald kan worden. Dit is natuurlijk niet noodzakelijk waar. Deze assumptie wordt ook gemaakt bij Markov counting. Modellen die de set van zones in zijn geheel als input nemen en een volgorde genereren maken deze veronderstelling niet en kunnen dus mogelijks beter presteren. Pointer netwerken blijken in Vinyals et al. (2015) goede resultaten te geven voor TSP's. Het probleem in deze context is dat er niet veel data ter beschikking is. Een voordeel is natuurlijk dat de gemiddelde lengte van een tour op het zone niveau van 22 is, wat relatief klein is. Het kan dus interessant zijn om te testen tot welke scores een pointer netwerk kan geraken en welk impact dat de hoeveelheid data erop heeft. Het is ook niet direct duidelijk hoe de zone afstanden mee in het model geïntegreerd zouden kunnen worden (als dit nog nodig zou zijn) zonder over de 5 minuten training tijd te gaan. Doordat de afgeleiden van het model complexer zijn is het ook niet duidelijk voor hoeveel epochs het getraind kan worden in 5 minuten.

5.4 Anders trainen

Uit sommige experimenten is het duidelijk da een betere validatie CE loss niet altijd vertaald wordt tot betere TSP resultaten. Dit was ook een veronderstelling die gemaakt werd. Leer methodes die gebruikt kunnen worden om dit te voorkomen zijn decision focussed learning of RL. Beide methodes integreren de TSP solver in het leerproces. Door dit te doen is de assumptie tussen CE loss en TSP resultaten niet meer nodig omdat het model uit een regret functie of rechtsreeks uit de zone score of AD kan leren. Dit creëert het probleem dat er geen zekerheid is dat het model goed genoeg kan leren in maximaal 5 minuten met een training proces die constant de TSP solver oproept. Het kan dus interessant zijn om te onderzoeken op welke manier de kwaliteit van de TSP solver een invloed heeft op de uiteindelijke volgorden. Er zou bijvoorbeeld een lager maximaal tijdsduur gezet kunnen worden op de TSP solver of zelfs met een andere heuristiek kunnen werken. In een extreem geval zou er verwacht kunnen worden dat het model utiliteiten genereert zodat een greedy TSP solver de zone volgorden heel snel kan reconstrueren. Er moet dus een trade-off gevonden worden zodat training tijd en kwaliteit van de training, en dus van de oplossingen.

Een andere leer methode die wel nog met CE loss werkt is om het EM algoritme te gebruiken om de initiële ideale kansen verdeling te relaxeren. Het is namelijk niet zo dat omdat een transitie gebeurt, dat ze dan een kans van 1 heeft en dat als een transitie niet gebeurt, dat ze dan een kans van 0 heeft. De echte kansenverdeling (als ze bestaat) zal zich er ergens tussen bevinden.

5.5 Aanpassingen voor challenge

Indien de geteste modellen gebruikt zouden moeten worden in de challenge zou het beter zijn om de training/validatie split anders aan te pakken. De 1107 validatie set van Cook et al. (2021) is beter dan de gebruikte 544 validatie set omdat het beter de verborgen test set representeert, het bevat namelijk geen routes waarbij zones meerdere keren worden bezocht. Langs de andere kant, zijn de gebruikte modellen gebaseerd op data en plots 563 hoge kwaliteit routes minder gebruiken kan wel een groot impact hebben op de scores. Om deze reden zou het best zijn om, in de context van de challenge, de hyperparameter tuning te doen met behulp van een k-fold crossvalidatie over de 1107 routes. Op deze manier kan zoveel mogelijk data gebruikt worden en is het model gevalideerd aan de hand van de set van routes die het meeste overeenkomt met de verborgen test set. Dit proces zou veel tijd in beslag nemen maar het kan op voorhand gebeuren. Het hoeft dus niet te tijdens de training fase van de challenge te gebeuren.

Hoofdstuk 6: Conclusie

Het twee staps algoritme van Canoy et al. (2022) heeft het potentieel om een stop score van 0.0079 te behalen, wat de Last Mile Routing Challenge praktisch zou oplossen. De beperkingen in de tweede stap zouden dus wel genoeg informatie bevatten om het probleem op te lossen. Verschillende manieren van Markov counting werden specifiek ontworpen voor de dataset van de challenge, om in dezelfde framewerk te blijven, en een referentie mechanisme om richtingen van zone volgorden te bepalen. De combinatie van deze methoden heeft geleid tot een stop score van 0.0276. Het is dus mogelijk om Markov counting op andere manieren toe te passen om patronen op verschillende niveaus te modelleren. Het referentie mechanisme lost het probleem van het bepalen van de richtingen bijna compleet op indien de perfecte zone volgorden worden gegenereerd.
Aanpakken die gebruik maken van lineaire combinaties hebben niet betere resultaten behaald omdat de modellen die gebaseerd zijn op Markov counting gemaakt zijn met datasets die subsets van elkaar zijn en complementeren elkaar dus niet. Verschillende linear layers gebruiken voor verschillende bronnen van transities is ook niet mogelijk voor dezelfde reden en omdat er niet genoeg data ter beschikking is om alle linear layers goed te kunnen trainen.
Een DNN om combinaties van Markov connectie kansen te benaderen heeft voor een stop score van 0.0257 gezorgd. Dit komt doordat de Markov connectie distributies veralgemeend en gesmooth kunnen worden en er wordt rechtsreeks informatie doorgegeven over de reële onderliggende distributie. Dit is helaas een indirecte aanpak en het is onduidelijk hoe de hyperparameters van het model efficiënt getuned kunnen worden. DNN's om ideale connectie en transitie kansen te benaderen hebben ook goede resultaten geleverd, namelijk 0.0265 en 0.027 respectievelijk, ook al konden richtingen niet goed genoeg bepaald worden door de ideale transitie kansen. De resultaten van de stop scores van de modellen liggen in de buurt van de resultaten van de winnende teams. Waarschijnlijk worden de beperkingen die door hun gemodelleerd zijn ook door de modellen gesimuleerd.
Het is dus mogelijk om utiliteiten van transities individueel te bepalen maar dit zou mogelijks de scores kunnen limiteren. Hetzelfde geldt ook voor het onafhankelijk bepalen van connectie volgorden en richtingen.
Designs van modellen die geconstrueerd zijn geweest door het Hyperband algoritme suggereren dat er meet preferenties zijn dan degene die gemodelleerd kunnen worden met algemene zone ID beperkingen.
Verder onderzoek kan zich focussen op het bepalen van richtingen te laten gebeuren bij de volgorde generatie aan de hand van pakketten informatie of voorrang penalty's. Andere architecturen en leer paradigma's zoals pointer netwerken, RL of EM kunnen getest worden om gemaakte veronderstellingen te doorbreken.

Bibliografie

Boost Graph Library: Transitive Closure - 1.55.0. (z.d.). Geraadpleegd op 5 juni 2023, van https://www.boost.org/doc/libs/1_55_0/libs/graph/doc/transitive_closure.html#:~:text=transitive_closure&text=The%20transitive%20closure%20of%20a,the%20transitive%20closure%20graph%20tc.

Canoy, R., Bucarey, V., Molenbruch, Y., Mulamba, M., Mandi, J., & Guns, T. (2022). Probability estimation and structured output prediction for learning preferences in last mile delivery. *arxiv.org*. https://arxiv.org/pdf/2201.10269.pdf

Canoy, R., & Guns, T. (2019). Vehicle Routing by Learning from Historical Solutions. In *Springer eBooks* (pp. 54–70). https://doi.org/10.1007/978-3-030-30048-7_4

Christofides, N. (2022). Worst-Case Analysis of a New Heuristic for the Travelling Salesman Problem. *Operations Research Forum*, *3*(1). https://doi.org/10.1007/s43069-021-00101-z

Cook, W., Held, S., & Helsgaun, K. (2022). Constrained Local Search for Last-Mile Routing. *arxiv.org*. https://arxiv.org/pdf/2112.15192.pdf

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms, third edition*. http://portal.acm.org/citation.cfm?id=1614191

Dantzig, G. B. (1963). Linear Programming and Extensions. In *RAND Corporation eBooks*. https://doi.org/10.7249/r366

Dantzig, G. B., & Ramser, J. H. (1959). The Truck Dispatching Problem. *Management Science*, *6*(1), 80–91. http://www.jstor.org/stable/2627477

Dorigo, M., & Stützle, T. (2010). Ant Colony Optimization: Overview and Recent Advances. In *Springer eBooks* (pp. 227–263). https://doi.org/10.1007/978-1-4419-1665-5_8

Gantenbein, D. (2023, 2 maart). *Winning Last Mile Challenge team addresses problem of combining mathematical routes with driver knowledge - Amazon Science*. Amazon Science. Geraadpleegd op 6 juni 2023, van https://www.amazon.science/academic-engagements/winning-last-mile-challenge-team-addresses-problem-of-combining-mathematical-routes-with-driver-

knowledge#:~:text=The%20winners%20of%20the%20Amazon,Germany%3B%20and%20Keld%20Helsgaun%2C%20associate

Guo, X., Mo, B., & Wang, Q. (2023). Amazon Last-Mile Delivery Trajectory Prediction Using Hierarchical TSP with Customized Cost Matrix. *arXiv*. https://arxiv.org/pdf/2302.02102.pdf

Hardesty, L. (2022, 13 juli). *Amazon, MIT team up to add driver know-how to delivery-routing models - Amazon Science*. Amazon Science. Geraadpleegd op 6 juni 2023, van https://www.amazon.science/blog/amazon-mit-team-up-to-add-driver-know-how-to-delivery-routing-models

Holland, J. L. (1992). Adaptation in Natural and Artificial Systems. In *The MIT Press eBooks*. https://doi.org/10.7551/mitpress/1090.001.0001

Jacobs, K., Rietra, M., Buvat, J., Cherian, S., Warner, S., Mazzo, L., Khadikar, A., & Khemka, Y. (2019). *The last-mile delivery challenge: Giving retail and consumer product customers a superior delivery experience without impacting profitability*. Capgemini Research Institute. https://www.capgemini.com/wp-content/uploads/2019/01/Report-Digital-%E2%80%93-Last-Mile-Delivery-Challenge1.pdf

Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by Simulated Annealing. *Science*, *220*(4598), 671–680. https://doi.org/10.1126/science.220.4598.671

Kleinberg, J., & Tardos, É. (2005). *Algorithm Design*. https://dl.acm.org/citation.cfm?id=1051910

Lin, S., & Kernighan, B. W. (1973). An Effective Heuristic Algorithm for the Traveling-Salesman Problem. *Operations Research*, *21*(2), 498–516. https://doi.org/10.1287/opre.21.2.498

Mandi, J., Canoy, R., Bucarey, V., & Guns, T. (2021). Data Driven VRP: A Neural Network Model to Learn Hidden Preferences for VRP. *CP, 2021*. https://arxiv.org/pdf/2108.04578.pdf

Merchán, D., Arora, J., Pachon, J., Konduri, K., Winkenbach, M., Parks, S., Noszek, J., & INFORMS. (2022). 2021 Amazon Last Mile Routing Research Challenge: Data Set. In *https://pubsonline.informs.org/doi/epdf/10.1287/trsc.2022.1173* (ISSN 0041-1655 (print), ISSN 1526-5447 (online)). INFORMS. Geraadpleegd op 25 februari 2023, van https://pubsonline.informs.org/doi/epdf/10.1287/trsc.2022.1173

MIT. (z.d.-a). *About the Challenge | Amazon Last-Mile Routing Research Challenge*. Geraadpleegd op 24 februari 2023, van https://routingchallenge.mit.edu/about-the-challenge/

MIT. (z.d.-b). *Amazon Last-Mile Routing Research Challenge | supported by the MIT Center for Transportation & Logistics*. Geraadpleegd op 25 februari 2023, van https://routingchallenge.mit.edu/

MIT. (2021, 10 augustus). *Last-mile routing challenge team performance and leaderboard.* | *Amazon Last-Mile Routing Research Challenge*. www.routingchallenge.mit.edu. Geraadpleegd op 2 maart 2023, van https://routingchallenge.mit.edu/last-mile-routing-challenge-team-performance-and-leaderboard/

MIT. (2022, 14 september). *Routing Challenge Dataset Now Publicly Available* | *Amazon Last-Mile Routing Research Challenge*. Geraadpleegd op 25 februari 2023, van https://routingchallenge.mit.edu/routing-challenge-dataset-now-publicly-available/

MIT CAVE Lab. (2021, 20 mei). *rc-cli/scoring at main · MIT-CAVE/rc-cli*. GitHub. Geraadpleegd op 26 februari 2023, van https://github.com/MIT-CAVE/rc-cli/tree/main/scoring

MIT Center for Transportation & Logistics. (2021, 8 maart). *Amazon Last Mile Routing Research Challenge First Informational Webinar* [Video]. YouTube. Geraadpleegd op 5 juni 2023, van https://www.youtube.com/watch?v=5JrASTtnNIk

Nagata, Y., & Kobayashi, S. (2013). A Powerful Genetic Algorithm Using Edge Assembly Crossover for the Traveling Salesman Problem. *INFORMS journal on computing*, *25*(2), 346–363. https://doi.org/10.1287/ijoc.1120.0506

Padberg, M., & Rinaldi, G. (1991). A Branch-and-Cut Algorithm for the Resolution of Large-Scale Symmetric Traveling Salesman Problems. *Siam Review*, *33*(1), 60–100. https://doi.org/10.1137/1033004

Statista. (2022). *eCommerce - Worldwide*. www.statista.com. Geraadpleegd op 25 februari 2023, van https://www.statista.com/outlook/dmo/ecommerce/worldwide

Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to Sequence Learning with Neural Networks. In *Neural Information Processing Systems* (Vol. 27, pp. 3104–3112). http://cs224d.stanford.edu/papers/seq2seq.pdf

Tarjan, R. E. (1972). Depth-First Search and Linear Graph Algorithms. *SIAM Journal on Computing*, *1*(2), 146–160. https://doi.org/10.1137/0201010

Traveling Salesperson Problem. (z.d.). Google for Developers. Geraadpleegd op 5 juni 2023, van https://developers.google.com/optimization/routing/tsp

University of Waterloo. (z.d.). *Traveling Salesman Problem*. Geraadpleegd op 5 juni 2023, van https://www.math.uwaterloo.ca/tsp/index.html

University of Waterloo. (2022). *JPT Optimization*. https://www.math.uwaterloo.ca/. Geraadpleegd op 27 februari 2023, van https://www.math.uwaterloo.ca/tsp/amz/code.html

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is All you Need. In *arXiv (Cornell University)* (Vol. 30, pp. 5998–6008). Cornell University. https://arxiv.org/pdf/1706.03762v5

Vinyals, O., Bengio, S., & Kudlur, M. (2016). Order Matters: Sequence to sequence for sets. In *arXiv (Cornell University)*. Cornell University. https://arxiv.org/pdf/1511.06391

Vinyals, O., Fortunato, M., & Jaitly, N. (2015). Pointer networks. In *Neural Information Processing Systems* (Vol. 28, pp. 2692–2700). https://papers.nips.cc/paper/5866-pointer-networks.pdf

Wang, S., Mo, B., & Zhao, J. (2020). Deep neural networks for choice analysis: Architecture design with alternative-specific utility functions. *Transportation Research Part C-emerging Technologies*, *112*, 234–251. https://doi.org/10.1016/j.trc.2020.01.012

Wikipedia contributors. (2023). Travelling salesman problem. In *Wikipedia*. Geraadpleegd op 5 juni 2023, van https://en.wikipedia.org/wiki/Travelling_salesman_problem

Winkenbach, M. (2021, 2 september). *Technical Proceedings of the Amazon Last Mile Routing Research Challenge*. https://dspace.mit.edu/handle/1721.1/131235